



PDF Download
3702974.pdf
30 December 2025
Total Citations: 1
Total Downloads: 714

Latest updates: <https://dl.acm.org/doi/10.1145/3702974>

RESEARCH-ARTICLE

FunFuzz: Greybox Fuzzing with Function Significance

RUIXIANG QIAN, Nanjing University, Nanjing, Jiangsu, China

QUANJUN ZHANG, Nanjing University, Nanjing, Jiangsu, China

CHUNRONG FANG, Nanjing University, Nanjing, Jiangsu, China

LIHUA GUO, Tsinghua University, Beijing, China

ZHENYU CHEN, Nanjing University, Nanjing, Jiangsu, China

Published: 07 May 2025
Online AM: 02 November 2024
Accepted: 28 September 2024
Revised: 10 August 2024
Received: 12 September 2023

[Citation in BibTeX format](#)

Open Access Support provided by:

Nanjing University

Tsinghua University

FuNFuzz: Greybox Fuzzing with Function Significance

RUIXIANG QIAN, QUANJUN ZHANG, and CHUNRONG FANG, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China
LIHUA GUO, School of Software, Tsinghua University, Beijing, China
ZHENYU CHEN, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

Greybox fuzzing is dedicated to revealing software bugs by maximizing code coverage. Concentrating on code coverage, greybox fuzzing effectively exposes bugs in real-world programs by continuously executing the program under test (PUT) with the test inputs generated from initial seeds, making it a popular software testing technique. Although powerful, the effectiveness of greybox fuzzing can be restricted in some cases. Ignoring the significant degrees of executed functions, traditional greybox fuzzing usually fails to identify *significant seeds* that execute more significant functions, and thus may assign similar energy to significant and trivial seeds when conducting power scheduling. As a result, the effectiveness of greybox fuzzing can be degraded due to wasting too much energy on trivial seeds. In this paper, we introduce *function significance* (FS) to measure the significant degrees of functions. Our key insight is that the influential functions that connect to many other functions are significant to greybox fuzzing as they provide more probabilities to reach previously unexplored code regions. To quantify FS, we conduct influence analysis upon the call graphs extracted from the PUTs to obtain the centrality values of function nodes. With FS as the significance measurement, we further propose FuNFuzz, an FS-aware greybox fuzzing technique, to optimize significant seeds and tackle the aforementioned restriction. To this end, FuNFuzz dynamically tracks the functions executed by a seed during fuzzing, and computes the significance score for the seed by accumulating the FS values of the functions executed by it. Based on the computed FS values, FuNFuzz then takes an estimation-based power scheduling to assign more (or less) energy to seeds that achieve over-estimated (or under-estimated) significance scores. Specifically, the seed energy is adjusted by multiplying with a scale factor computed regarding the ratio of the *actual significance score* achieved by executing the seed and the *estimated significance score* predicted by a linear model constructed on-the-fly. To evaluate FuNFuzz, we prototype it on top of AFL++ and conduct experiments with 15 programs, of which 10 are from common real-world projects and five are from Magma, and compare it to seven popular fuzzers. The experimental results obtained through fuzzing exceeding 40,800 CPU hours show that: (1) In terms of covering code, FuNFuzz outperforms AFL++ by achieving 0.1%–18.4% more region coverage on 13 out of 15 targets. (2) In terms of finding bugs, FuNFuzz unveils 114 unique crashes and 25

This work was partially done when Lihua Guo was at Nanjing University.

This work was supported partially by the National Natural Science Foundation of China (61932012, 62372228), the CCF-Huawei Populus Grove Fund (CCF-HuaweiSE202304, CCF-HuaweiSY202306), the Science, Technology and Innovation Commission of Shenzhen Municipality (CJGJZD20200617103001003), and the Fundamental Research Funds for the Central Universities (14380029).

Authors' Contact Information: Ruixiang Qian, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: qianrx@smail.nju.edu.cn; Quanjun Zhang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: quanjun.zhang@smail.nju.edu.cn; Chunrong Fang (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: fangchunrong@nju.edu.cn; Lihua Guo, School of Software, Tsinghua University, Beijing, China; e-mail: glh22@mails.tsinghua.edu.cn; Zhenyu Chen, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zychen@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://doi.org/10.1145/3702974).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/4-ART99

<https://doi.org/10.1145/3702974>

Magma bugs (which are derived from CVEs) in 20 trials of 24-hour fuzzing, which are the most compared to the competitor fuzzers and include 32 crashes and 1 Magma bug that the other fuzzers fail to discover. Besides the experiments focusing on code coverage and bug finding, we evaluate the key components of FUNFUZZ, namely the FS-centered estimation-based power scheduling and the lazy FS computation mechanism. The extensive evaluation not only suggests FUNFUZZ's superiority in code coverage and bug finding, but also demonstrates the effectiveness of the two components.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Theory of computation** → **Program analysis**;

Additional Key Words and Phrases: Greybox Fuzzing, Function Significance, Influence Analysis

ACM Reference format:

Ruixiang Qian, Qianjun Zhang, Chunrong Fang, Lihua Guo, and Zhenyu Chen. 2025. FUNFUZZ: Greybox Fuzzing with Function Significance. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 99 (April 2025), 34 pages. <https://doi.org/10.1145/3702974>

1 Introduction

Fuzzing [54], or fuzz testing is one of the most powerful testing techniques, which has been widely adopted due to its easy-to-deployment nature and bug-finding capability [50, 74, 85]. Greybox fuzzing is an important family of existing fuzzing techniques, which is popular for the balance it makes between the effectiveness and efficiency [31, 59, 70]. Given a **program under test (PUT)** and several initial seed inputs, a greybox fuzzer (the tool that implements greybox fuzzing) first instruments the PUT to collect code coverage at runtime. It then repeatedly selects seeds from the seed queue and generates new test inputs from the selected ones to continuously execute the PUT [47, 65]. Once fresh coverage is observed, the generated test inputs are added to the seed queue for subsequent fuzzing. To steer executions to rarely explored code regions, greybox fuzzing leverages code coverage, such as edge coverage, as feedback to schedule the energy (i.e., time budget) assigned to seeds to control test executions [23, 38]. This step is typically termed power scheduling in prior research [10]. Generally, greybox fuzzing favors seeds that achieve more coverage and assigns more energy to them to generate more test inputs from them in order to keep their merits in maximizing code coverage [14]. The above heuristics make greybox fuzzing a simple yet effective technique, which has discovered thousands of bugs from hundreds of real-world programs [81].

Although the effectiveness of greybox fuzzing has been demonstrated by both academia and industry, it can still be restricted in some cases [53, 69]. One possible reason for such latent restriction is that traditional greybox fuzzing merely concentrates on code coverage while ignoring the characteristics, say significant degrees, of the functions under test. The significant degrees of functions may vary facing different software engineering tasks. For example, **directed greybox fuzzing (DGF)** considers the functions closer to the preset target sites to be more significant, since the main goal of DGF is to reach the target sites as fast as possible [10]. A code clone detection technique may treat a function containing functional clones as more significant, as such clones are tricky to detect [21]. Similarly, when designing a general-purposed greybox fuzzing technique, what types of functions should be attached with great significance? Given a PUT represented as a call graph where the nodes denote the functions and the edges denote the call relations, *the influences of functions* are greatly meaningful to greybox fuzzing as they reflect the probabilities of reaching other callee functions through executing a certain caller. In this paper, we focus on such influences and employ them as the **function significance (FS)**. Table 1 displays some call relations extracted from a real-world project MJS [55], where the functions `parse_if` and `parse_return` are two callees of the function `parse_statement`. The function `parse_if` is more

Table 1. Call Relations Extracted from the Real-World Project MJS

Caller	Callee	First Occur
parse_statement (total: 27)	parse_return parse_if ...	mjs.c:13329 mjs.c:13344 ...
parse_if (total: 27)	cs_log_print_prefix	mjs.c:12893
	cs_log_printf	mjs.c:12893
	mjs_set_errorf	mjs.c:12894
	pnext	mjs.c:12894
	parse_expr	mjs.c:12896
	emit_byte	mjs.c:12898
	emit_init_offset	mjs.c:12898
	parse_block_or_stmt	mjs.c:12903
	mjs_bcode_insert_offset	mjs.c:12920
...	...	...
parse_return (total: 7)	mjs_set_errorf	mjs.c:12941
	cs_log_print_prefix	mjs.c:12941
	cs_log_printf	mjs.c:12941
	pnext	mjs.c:12941
	parse_expr	mjs.c:12941
	emit_byte	mjs.c:12942
...	...	...

The first and second columns are the declaring names of caller and callee functions. The third column is the first occurrence of call relations in “File:Line” format. The “total” displays the total number of the callees that a caller function connects to.

influential than `parse_return` as it connects to more callee functions (i.e., 27 vs. 7). Intuitively, executing functions like `parse_if` is more likely to reach the code regions that reside in previously unexplored functions, suggesting that such functions should be attached with higher significance. As for (mutational) greybox fuzzing, seeds are critical to its overall effectiveness, as they lay the foundation of input generation and decide the direction of test executions [63]. As such, seeds that execute more significant functions deserve more energy (i.e., the time budget allocated for the seeds to spawn offspring inputs) at power scheduling, as they have more potential to reach fresh code regions. To understand seeds’ capabilities in exercising significant functions, we conduct a preliminary study by fuzzing a real-world program `objdump` for 24 hours with `AFL++` [23], and quantifying such capabilities as significance scores by accumulating the FS values of the functions covered by seeds. Figure 1 visualizes the distribution of the significance scores obtained, where the Y-axis represents the significance scores achieved by seeds and the X-axis represents the edge coverage. As shown in Figure 1, instead of gathering in a small scope, the significance scores at the edge coverage of 600 spread over in a relatively large range from 1.75 to 2.15. A phenomenon like this showcases that the seeds’ capabilities in exercising significant functions are quite unequal in practice. Unfortunately, traditional greybox fuzzers are generally *unaware* of such inequality. They usually treat all functions the same and fail to distinguish the promising seeds that are good at executing significant functions, which may consequently restrict their effectiveness.

To mitigate the aforementioned issues, we propose FUNFUZZ, an FS-aware greybox fuzzing technique to facilitate fuzzing via optimizing significant seeds that achieve higher significance

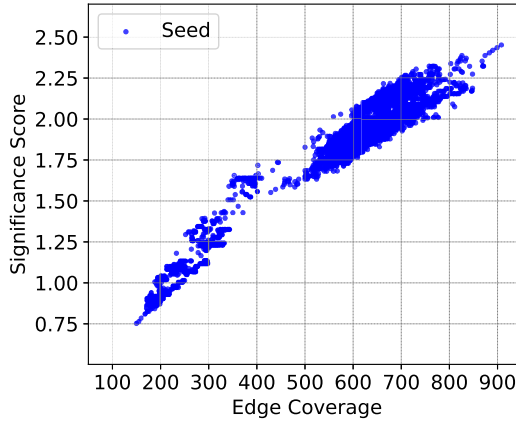


Fig. 1. Significance scores achieved by seeds when fuzzing objdump with AFL++ for 24 hours.

scores. The features of FUNFuzz can be vaguely concluded as three parts: (1) *quantifying the significant degrees of functions*, (2) *identifying significant seeds*, and (3) *optimizing significant seeds*. To *quantify the significant degrees of functions*, we extract the call graph from the PUT, calculate edge weights with the call probabilities among functions, and conduct influence analysis [32, 72] upon the extracted call graphs to calculate FS values (Section 4.1). Note that we reverse the directions of calls at influence analysis to propagate influence to caller functions at dominant positions. To *identify significant seeds* that exercise more significant functions, we instrument the call sites of the PUT, dynamically track the executed functions at runtime, and score the queued seeds by accumulating the FS values of the functions executed by them. To *optimize significant seeds*, we propose an FS-centered estimation-based power scheduling to assign more (or less) energy to seeds that achieve over-estimated (or under-estimated) significance scores (Section 4.3). The estimation of significance scores is realized by a simple linear model built upon the insights extracted from an empirical study, where the independent variable is the edge coverage and the dependent variable is the significance score. Take a simple yet concrete example with Figure 1, the seed that achieves a significance score of 2.15 at the coverage of 600 will be rewarded with zoomed-up energy, as it achieves an over-estimated significance score regarding the constructed model, while the seed that achieves a significance score of 1.75 will be punished with zoomed-down energy, as it behaves worse than the estimation of the model.

We prototype FUNFuzz on top of the **state-of-the-art (SOTA)** fuzzer AFL++ [23] and conduct a large-scale experiment with 15 fuzz targets for evaluation, where 10 targets are from commonly used real-world projects and the other five are from a mature bug-based fuzzing benchmark named Magma [27]. We compare FUNFuzz to seven fuzzers, each of which is either a representative SOTA fuzzer (e.g., Entropic and AFL++) or adopts a similar strategy to FUNFuzz (e.g., FairFuzz and K-Scheduler). The results obtained through the experiments exceeding 40,800 CPU hours show that: (1) In terms of edge coverage, FUNFuzz achieves 0.1%–18.4% more region coverage than AFL++, which is not only the implementation base of FUNFuzz but also a SOTA fuzzer, on 13 out of 15 targets, and performs the best on 12 out of 15 targets compared to all studied fuzzers. (2) In terms of finding bugs, FUNFuzz finds 114 unique crashes from the 10 real-world fuzz targets and triggers 25 Magma bugs, which are the most compared to all the studied fuzzers; it also exposes 32 crashes and 1 Magma bug that the other competitor fuzzers fail to discover. In addition to the contrast experiments, we conduct ablation experiments to assess the effectiveness of the key components of FUNFuzz, namely the FS-centered estimation-based power scheduling and the lazy FS computation

mechanism. The experimental results reveal that: (1) The FS-centered estimation-based power scheduling strategy helps FUNFUZZ cover 0.4%–19.1% more region coverage compared to a strategy estimating with only edge coverage. (2) The lazy FS computation mechanism successfully alleviates the extra overhead brought by updating FS values, letting fuzzing throughput not be impacted by the update period. To sum up, the contributions of this paper are as follows:

- *Preliminary study.* We perform a preliminary study to investigate the phenomenon of the inequality of FS residing in real-world programs. To our knowledge, this is the first study assessing the significant degrees of functions from the perspective of greybox fuzzing.
- *Innovative measure.* Based on the preliminary study, we propose FS, an innovative measurement based on influence analysis to quantify the significant degrees of the functions.
- *Novel scheduling.* We build a simple linear mode to estimate significance scores for seeds and propose an estimation-based power scheduling to facilitate greybox fuzzing by optimizing seeds that achieve over-estimated significance scores and deoptimizing the under-estimated ones.
- *Extensive study.* We conduct well-designed experiments involving eight fuzzers and 15 real-world programs to evaluate FUNFUZZ in terms of code coverage and bug-finding capability. We also conduct ablation studies to evaluate the key components of FUNFUZZ and discuss FUNFUZZ’s capability in handling indirect calls. The extensive study not only demonstrates the effectiveness of FUNFUZZ but also profoundly explores our technical choices.
- *Practical tool.* We implement FUNFUZZ on top of AFL++ [23] and release all artifacts of FUNFUZZ (i.e., the datasets and source code) to support open science [24].

2 Background

2.1 Greybox Fuzzing

Greybox fuzzing endeavors to expose bugs by maximizing coverage. It monitors execution statuses with coverage collected by lightweight instrumentation and drives executions towards rarely explored code regions. Algorithm 1 shows the general process of greybox fuzzing. Given a set of initial seeds S and an instrumented PUT p , greybox fuzzing repeatedly generates new test input s' from the selected seed s and executes s' against p to bind it with achieved coverage (lines 7–8). According to the execution result res , s' can be next appended to (1) crashing seed queue S^* if it induces any crashes; or (2) interesting seed queue S' if it attains fresh coverage. The main fuzzing loop (lines 3–15) continues to work until resources are run out or an abort-signal is received.

To steer fuzzing towards the expected direction, the selected seed s is scheduled at the beginning of the fuzzing loop to get energy e for generating new test inputs (lines 5–6). The process of assigning energy to seeds is termed as power scheduling in prior research [9, 80]. Generally, greybox fuzzing favors seeds that achieve more coverage and assigns more energy to them to keep the capabilities in maximizing coverage [14]. In this paper, we additionally concentrate on the significant degrees of functions. Concisely, we score the seeds with the FS values of the executed functions and amplify their energy regarding the achieved significance scores via power scheduling. We describe the details of the FS-centered estimation-based power scheduling in Section 4.3.

2.2 Influence Analysis

Influence analysis measures the centrality degree of nodes and identifies the most influential ones for the given graph [40]. Various types of centrality measures have been proposed to quantify the influence of nodes with different focuses, including degree centrality, closeness centrality, betweenness centrality, and eigenvector centrality [58]. The eigenvector centrality provides global influence over the entire graph by quantifying the importance of a node according to the importance

Algorithm 1: Greybox Fuzzing**Input:** Initial Seeds S , Instrumented PUT p **Output:** Interesting Seeds S' , Crashing Seeds S^*

```

1:  $S' \leftarrow S$ 
2:  $S_{crash} \leftarrow \emptyset$ 
3: repeat ▷ Main fuzzing loop
4:    $s \leftarrow \text{CHOOSENEXT}(S')$ 
5:    $e \leftarrow \text{ASSIGNENERGY}(s)$  ▷ Power scheduling
6:   for  $i$  from 1 to  $e$  do
7:      $s' \leftarrow \text{GENERATENEWINPUT}(s)$ 
8:      $res \leftarrow \text{EXECUTE}(p, s')$ 
9:     if  $\text{ISCRASH}(s', res)$  then
10:      add  $s'$  to  $S^*$ 
11:     else if  $\text{ISINTERESTING}(s', res)$  then
12:      add  $s'$  to  $S'$ 
13:     end if
14:   end for
15: until resources exceeds or abort-signal comes
16: return  $S', S^*$ 

```

of the neighbors of the node [62, 72], which fits our goal of identifying significant functions for a given call graph. In this paper, we adopt Katz centrality, a variant of eigenvector centrality, to capture the significant degrees of functions. The superiority of Katz in tackling fuzzing-relevant tasks has been demonstrated by She et al. [72].

Calculating Katz Centrality. Katz centrality is first introduced by Leo Katz in 1953 to measure the relative degree of influence of a node within a given graph [32]. Suppose $G = \langle V, E \rangle$ is a directed graph with n nodes $V = \{v_1, v_2, \dots, v_n\}$, the target of the Katz algorithm is to find a vector of stationary distributed centrality values $\mathbf{c}$, which can be formalized as:

$$\mathbf{c} = \alpha \mathbf{A} \mathbf{c} + \boldsymbol{\theta}, \quad (1)$$

where $\mathbf{A}$ is the adjacency matrix of G , $\boldsymbol{\theta}$ is the vector of the minimum centrality values of the nodes, and $\alpha \in [0, 1]$ is the damping factor for penalizing connections with distant neighbors [3]. Locally, the centrality value c_i of an individual node v_i is determined by its neighbors, which can be formalized as:

$$c_i = \alpha \sum_{j=1}^n A_{i,j} c_j + \theta_i, \quad (2)$$

where c_j is the centrality value of node v_j , $A_{i,j}$ is the weight of the edge $e = \langle v_i, v_j \rangle$, and θ_i the minimum centrality value of node v_i . In this article, the directed graph we analyze is the call graph of the PUT and the nodes we concentrate on are functions. We calculated FS values by conducting the Katz algorithm upon given call graphs, of which the process is described in Section 4.1.3.

2.3 Estimation with Simple Linear Models

Linear models are widely used for estimation tasks in various fields such as machine learning, econometrics, physical science, and social science [22, 51, 76]. Simple linear models are one of the most widely used linear models that attempt to model the relationship between two variables [66].

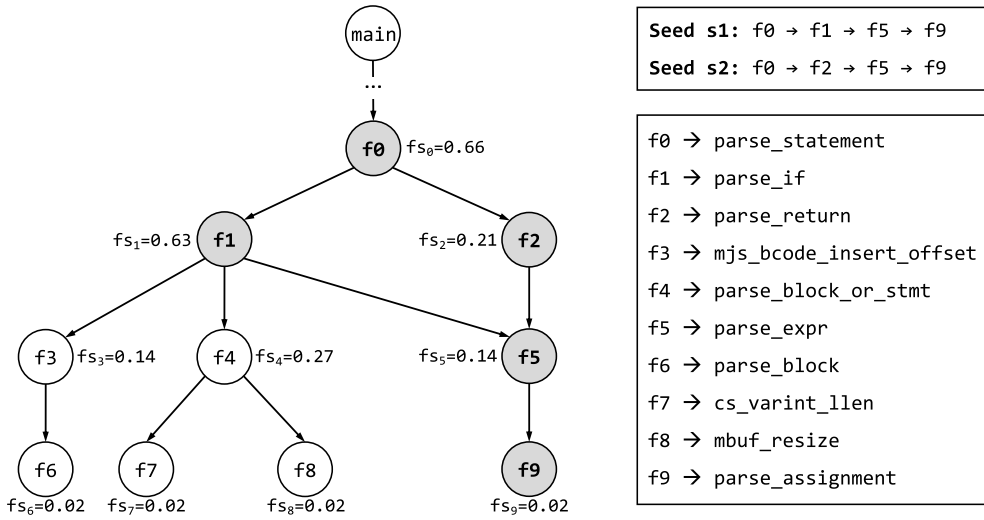


Fig. 2. An exemplified call graph extracted from MJS. [55].

Let $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ be a dataset with n ($n \in \mathbb{N}^*$) observations, the simple linear regression model for $\mathcal{D}$ can be established as:

$$y = \beta_0 + \beta_1 x + \epsilon, \quad (3)$$

where y is the dependent variable, x is the independent variable, ϵ is the random variable that represents measurement errors, and β_0, β_1 are regression parameters. We treat the edge coverage and significance scores achieved by seeds as independent and dependent variables and construct a simple linear to estimate the significance scores the seeds should achieve during fuzzing. With the constructed linear model, we propose an estimation-based power scheduling to optimize seeds that achieve over-estimated significance scores. We elaborate on the scheduling strategy in Section 4.3.

3 A Motivating Example

In this section, we present the core insights of FUNFUZZ with a simple example extracted from project MJS [55]. Figure 2 displays the exemplified call graph consisting of 10 function nodes $f0 \sim f9$. The detailed corresponding table from function nodes to real MJS functions are shown at the right bottom of Figure 2. The “main” node represents the entry point of execution, and the directed edges between nodes represent the call relations between functions. The peak function $f0$ connects to $f1$ and $f2$ via two call relations $f0 \rightarrow f1$ and $f0 \rightarrow f2$, which respectively corresponds to the real call relations “parse_statement $\rightarrow$ parse_if” and “parse_statement $\rightarrow$ parse_return”. Recall Table 1, the callee functions that can be reached through $f1$ (i.e., $\{f3, f4, f6, f7, f8\}$) are more abundant than the ones through $f2$ (i.e., $\{f5, f9\}$), reflecting that parse_if connects to more callees (27 in total) than parse_return does (seven in total).

The drawback of traditional greybox fuzzing. Hoping to preserve merits in maximizing coverage [14], existing greybox fuzzers like AFL++ [23] usually score the seeds according to the coverage they achieve at executing the PUT and assign more energy to the ones with higher scores at power scheduling. As shown in Figure 2, $s1$ and $s2$ are two seeds that execute two different call chains “ $f0 \rightarrow f1 \rightarrow f5 \rightarrow f9$ ” and “ $f0 \rightarrow f2 \rightarrow f5 \rightarrow f9$ ”. For simplicity, suppose both seeds achieve an edge coverage of 400 (100 for each executed function) at their individual executions. When conducting power scheduling, AFL++ treats $s1$ and $s2$ equally as they both achieve scores of 400 at their last

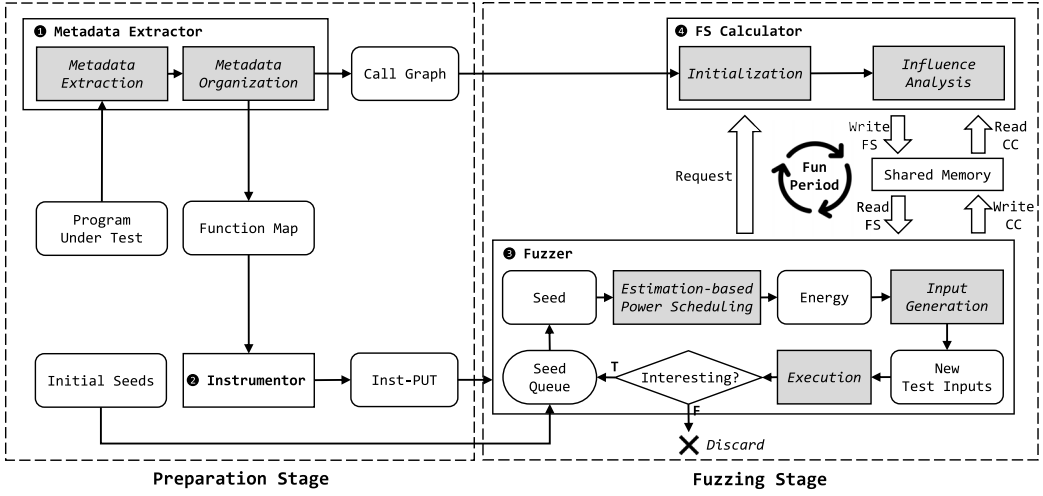


Fig. 3. The architecture of FUNFuzz, where the components are marked in bold and tagged with serial numbers, the operations are presented in italic fonts with gray backgrounds, and the input and output objects are put in round rectangles. The solid arrows represent the sequence of operations whereas the hollow arrows represent the asynchronous operations performed in every “Fun Period.”

executions and thus will assign similar energy to them. Such scheduling is inappropriate as the function f_1 that the seed s_1 executes is in fact more influential than the function f_2 that the seed s_2 executes. More energy should be assigned to s_1 as it is more likely to reach previously unexplored functions, such as f_3 and f_4 . However, ignoring the differences among the significant degrees of the executed functions, traditional greybox fuzzers fail to optimize significant seeds like s_1 and may waste too much energy on trivial seeds like s_2 .

Our solution. We propose FUNFuzz to facilitate greybox fuzzing via optimizing seeds like s_1 . To this end, we first introduce FS to measure the significant degrees of functions. The FS values of functions are obtained by analyzing the influences of functions within the given call graph. Conceptually, the FS value of a function relies on (1) the FS values of the callees the function connects to and (2) the closeness of the relationships between the function and its callees, which are captured by edge weights. In this example, we assume all edge weights are 1 for simplicity. Before calculating FS values for functions via influence analysis, we first reverse the directions of edges of the given call graph to propagate centrality values to caller functions. Taking the Katz algorithm with damping factor $\alpha = 0.5$ and minimum centrality $\theta = 0.1$ for influence analysis, the FS values of the functions are calculated as the annotations of nodes in Figure 2. Through FS values, one can easily find that f_1 , which achieves an FS value of 0.63, is far more significant than f_2 , which only achieves an FS value of 0.21. We then adjust the scores of seeds with the summary of FS values achieved by the executed functions, where the score of s_1 is reshaped to $580 = 400 \times (0.66 + 0.63 + 0.14 + 0.02)$, and the score of s_2 is to $412 = 400 \times (0.66 + 0.21 + 0.14 + 0.02)$. With the adjusted score, FUNFuzz can identify significant seeds like s_1 and optimize them by assigning more energy at the power scheduling step.

4 FUNFUZZ

In this section, we elaborate on the methodology of FUNFuzz. Figure 3 exhibits the architecture of FUNFuzz, which comprises four components (i.e., a metadata extractor, an instrumentor, a fuzzer, and an FS calculator). Given a PUT and a set of initial seeds, FUNFuzz operates in two stages: the

preparation stage and the *fuzzing stage*. The metadata extractor and the instrumentor work at the preparation stage whereas the FS calculator and the fuzzer work at the fuzzing stage.

Preparation stage. The main goal of this stage is to collect program metadata needed for the fuzzing stage. To this end, FUNFUZZ first analyzes the PUT with the metadata extractor to acquire raw program metadata (i.e., function names and call relations) and then assigns each function name a unique ID to build a bijective map between function names and function IDs. Based on the extracted call relations, a call graph is constructed to serve as the foundation of FS computation. Based on the established function map, the PUT is instrumented by the instrumentor to count for occurred calls at runtime. Finally, the function map, constructed call graph, and instrumented PUT are passed to the fuzzing stage along with the initial seeds for further usage.

Fuzzing stage. At this stage, FUNFUZZ performs an FS-aware fuzzing with the fuzzer and FS calculator. It first setups the FS calculator with the call graph before the main fuzzing loop kicks off. During the fuzzing loop, the fuzzer continuously executes the PUT to collect code coverage and **call counts (CC)** (i.e., the number of times a call relation happens). The fuzzer passes the CC to the FS calculator for every δ seconds and asks for FS values. We refer to δ as “*Fun Period*.” Once the fresh CC are received, the FS calculator updates the edge weights of the call graph and calculates FS values using the Katz algorithm. The update of FS values is described in Section 4.1. We adopt a lazy strategy to minimize the overhead brought by dynamic FS computation. Specifically, we use the FS values from the last computation that are stored in steady memory for FS-aware fuzzing, and let the FS calculator write the updated FS values to another area of memory. The implementation details of the lazy strategy are described in Section 5.3. With the computed FS values, FUNFUZZ calculates the significance score for each seed before adding it to the queue. The calculation of the significance scores is described in Section 4.2. To optimize seeds that execute significant functions, we come up with an FS-centered estimation-based scheduling strategy to assign more (or less) energy to seeds that have achieved significance scores over (or under) the estimation. The FS-centered estimation-based scheduling is discussed in Section 4.3.

4.1 Updating FS Values

FUNFUZZ asks the FS calculator to update FS values with the CC at the end of each fun period. Algorithm 2 illustrates the full process of calculating FS values. Specifically, the FS calculator takes three steps to update FS values with fresh CC: (1) updating global CC, (2) updating edge weights, and (3) calculating FS values.

4.1.1 Updating Global CC. FUNFUZZ leverages global CC (i.e., the CC that emerged since fuzzing started) to compute FS values. To this end, the FS calculator maintains a $(n + 1) \times (n + 1)$ adjacency matrix to record global CC (line 2), where n is the number of functions under test. There are two types of counts that are updated at each computation, namely $C_{i,j}$ and C_i (lines 4–9). Specifically, $C_{i,j}$ stands for the number of times that the call relation $\langle f_i, f_j \rangle$ appears, and C_i stands for the number of times that f_i performs as a caller. The CC $C_{i,j}$ and caller counts C_i satisfy the following equation:

$$C_i = \sum_{j=1}^n C_{i,j}. \quad (4)$$

4.1.2 Updating Edge Weights. The FS calculator updates the edge weights of the call graph after refreshing the global CC (lines 10–15). An edge weight typically represents the strength of the relationship between two connected nodes in the network analysis [44], and the influences of nodes tend to flow through edges with higher weights. In our cases, the nodes of the analyzed network are functions under test, and the edges between nodes are the call relations. To optimize seeds that

Algorithm 2: FS Values Calculation

```

1: global variables
2:   Global Call Counts  $C$ , initialized as all 0
3: end global variables
Input: Fresh Call Counts  $C'$ , Call Graph  $G$ , Number of Functions  $n$ 
Output: Updated FS values  $V$ 
4: for  $i$  from 1 to  $n$  do ▷ Updating global call counts
5:   for  $j$  from 1 to  $n$  do
6:      $C_{i,j} \leftarrow C_{i,j} + C'_{i,j}$ 
7:      $C_i \leftarrow C_i + C'_{i,j}$ 
8:   end for
9: end for
10: for  $i$  from 1 to  $n$  do ▷ Updating edge weights
11:   for  $j$  from 1 to  $n$  do
12:      $w \leftarrow \text{COMPUTEWEIGHT}(C, i, j)$ 
13:      $\text{UPDATEWEIGHT}(G, i, j, w)$ 
14:   end for
15: end for
16:  $V \leftarrow \text{INFLUENCEANALYSIS}(G)$  ▷ Calculate FS values
17: return  $V$ 

```

trigger rare call relations, we compute edge weights with the complementary call probabilities of every two connected functions. Specifically, for two functions f_i and f_j that are connected with a call relation $e_{i,j} = \langle f_i, f_j \rangle$, the call probability $p_{i,j}$ between them is computed as the ratio of the CC $C_{i,j}$ and the caller counts C_i (i.e., $p_{i,j} = \frac{C_{i,j}}{C_i}$). Suppose the number of functions under test is n , the complementary call probability between f_i and f_j is then computed as $\sum_{j=1}^n (p_{i,j}) - p_{i,j} = \frac{\sum_{j=1}^n C_{i,j}}{C_i} - p_{i,j} = 1 - p_{i,j}$ regarding Equation (4).

However, there are two special cases that we need to take into consideration. Firstly, for connected functions that have call relations statically identified at the preparation stage but whose call probabilities are 0, it is inappropriate to assign their weights to 1 as their call relations have never appeared at runtime so far. Secondly, for connected functions whose call probabilities are 1, it is inappropriate to assign their weights to 0 as they are in fact related. What's worse, 0 weights can fail the influence analysis in practice. We tackle the two cases leveraging the minimum weight w_0 ($w_0 \rightarrow 0$). Specifically, for connected functions whose call probabilities are 0, we set the edge weights to w_0 ; while for the ones whose call probabilities are larger than 0, we compute the edge weights by adding the complementary call probabilities with w_0 . Above all, the edge weight $w_{i,j}$ of the call relation $e_{i,j}$ is computed as:

$$w_{i,j} = \begin{cases} w_0 & \text{if } p_{i,j} = 0 \\ 1 - p_{i,j} + w_0 & \text{if } p_{i,j} > 0 \end{cases} \quad (5)$$

4.1.3 Calculating FS Values with Katz. We leverage the Katz algorithm to calculate FS values for functions. Generally, a call graph is a directed graph with edges (i.e., the call relations) pointing from the caller functions to the callee functions. Therefore, to propagate centrality values to significant functions that call and connect to other functions, we first reverse the directions of edges when conducting the Katz algorithm. Take edge $e_{i,j} = \langle f_i, f_j \rangle$ as an example, its direction is considered $f_j \rightarrow f_i$ in influence analysis. Suppose there are n functions in the given call graph. Let $\text{sig}(f_i)$ be

the FS value of function f_i and $w_{i,j}$ be the weight of edge $e_{i,j}$. With Equation (2), we have:

$$\text{sig}(f_i) = \alpha \sum_{j=1}^n w_{i,j} \text{sig}(f_j) + \theta_i. \quad (6)$$

4.2 Calculating Significance Scores

FUNFUZZ exploits FS values obtained from the FS calculator to compute significance scores for seeds before adding them to the queue. Significance scores are later used in power scheduling to optimize seeds that execute more influential functions. Specifically, FUNFUZZ tracks the execution statuses of functions at runtime and accumulates the FS values of the executed functions to compute significance scores for seeds. Suppose there are n functions under test and denote the execution status mapping as E , then the execution status of a function f_i regarding a seed s can be denoted as $E(s, f_i)$, where $E(s, f_i) = 1$ if function f_i is executed by the s , else $E(s, f_i) = 0$. Let $\text{sig}(f_i)$ be the FS value of f_i , the significance score $\text{sig}(s)$ of the seed s is computed as:

$$\text{sig}(s) = \sum_{i=1}^n E(s, f_i) \text{sig}(f_i). \quad (7)$$

4.3 Estimation-Based Power Scheduling

The main insight of FUNFUZZ is to facilitate greybox fuzzing by allocating more resources to promising seeds that execute more significant functions. To this end, we propose an estimation-based power scheduling to reward (or punish) seeds that achieve over-estimated (or under-estimated) significance scores, which is realized with a simple linear model [66]. This design is enlightened by the coverage-based mechanism adopted by AFL++, in which a seed's energy is adjusted according to the ratio of the coverage achieved by the seed and the average coverage achieved by all seeds [23]. In the rest of this section, we first introduce the linear model used for estimating significance scores for seeds and then elaborate on the process of estimation-based power scheduling.

4.3.1 Estimation Model and Its Feasibility. FUNFUZZ estimates the normalized significance score that a seed should achieve at runtime with a simple linear model $y = \mu x$, where y represents the estimated normalized significance score of a seed and x represents the real normalized coverage the seed has achieved. The estimation coefficient μ is calculated as the ratio of the average normalized significance score to the average normalized coverage (i.e., $\mu = \bar{y}/\bar{x}$). We present the feasibility of estimating significance scores with the model $y = \mu x$ by answering the following questions

- *Why linear model?* In intuition, the significance scores and coverage achieved by seeds are naturally correlated in a positive manner mainly for two reasons. First, in our design, a seed's significance score is calculated by accumulating the FS values of the functions that are executed by the seed (Equation (7)). Second, seeds that execute more functions tend to cover more code. To further analyze the correlation, we conduct a preliminary study on 10 real-world targets. We first fuzz each target with a modified AFL++ [23], which is FS-aware but with its builtin power scheduling, for 24 hours to collect the achieved coverage and significance scores of the queued seeds, and then calculate the **Pearson Correlation Coefficient (PCC)** between coverage and significance scores. PCC measures the strength of the linear relationship between two variables [18]. It takes a value from -1 to 1 , where 0 means no correlation, 1 means perfect positive correlation, and -1 means perfect negative correlation [34]. Table 2 displays the PCC values between seeds' coverage and significance scores at PCC columns. Generally, a PCC value that is larger than 0.7 indicates a strong to very strong positive linear correlation between the two studied variables [4]. The empirical results show that significance

Table 2. Results of Analyzing on Seeds' Significance Scores and Coverage

Target	PCC	Norm. R^2	Orig. R^2	Target	PCC	Norm. R^2	Orig. R^2
cxxfilt	0.92	0.54	0.44	objdump	0.97	0.92	0.73
djpeg	0.99	0.97	0.85	readelf	0.90	0.54	0.53
mjs	0.88	0.56	0.67	readpng	0.99	0.96	0.70
mutool	0.96	0.91	0.88	tcpdump	0.71	0.30	0.18
nm	0.97	0.93	0.91	xmllint	0.93	0.82	0.13

scores achieved by seeds are linearly correlated with edge coverage achieved by seeds in all 10 targets, supporting us using a linear model for estimation.

- *Why not OLS model?* In simple linear regression, the OLS model is typically considered the line of best fit as it has the least variance [76]. To build such a model, the OLS method is used to calculate the parameters β_0 and β_1 regarding Equation (3) as:

$$\beta_1 = \frac{\sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad (8)$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}, \quad (9)$$

where $\bar{x}$ is the mean of $X = \{x_1, x_2, \dots, x_n\}$ and $\bar{y}$ the mean of $Y = \{y_1, y_2, \dots, y_n\}$. We do not adopt an OLS model for estimation mainly out of efficiency. In our cases, X is the set of edge coverage achieved by seeds whereas Y is the set of significances. The set size n corresponds to the size of the seed queue, which is continuously growing during fuzzing. To calculate β_0 and β_1 at runtime, the fuzzer needs to first calculate the differences between every observed value (i.e., x_i and y_i) and the mean value (i.e., $\bar{x}$ and $\bar{y}$), which consequently induces great overhead. Therefore, to make a tradeoff between the fitness of the estimation model and the efficiency of fuzzing, we assume the parameter β_0 as 0 and calculate β_1 (i.e., the estimation coefficient μ) as $\bar{y}/\bar{x}$ to build the simple linear model $y = \mu x$ instead of using the OLS method.

- *Why normalized data and how they fit the model?* In our cases, estimation with normalized data is fitter than that with original data, as the normalized data include the point (0, 0) that the model $y = \mu x$ gets through whereas the original data may usually exclude. Therefore, it may be inappropriate to calculate the estimation coefficient as $\mu = \bar{y}/\bar{x}$ for some PUTs if the original data are considered for estimation. Figure 4 visualizes the extent to which the normalized data (Figure 4(a)) and the original data (Figure 4(b)) fit the model $y = \mu x$ using the data generated by fuzzing objdump with AFL++ for 24 hours. The black lines represent the OLS models while the red lines represent $y = \mu x$. Compared to the original data, the normalized data fit $y = \mu x$ better as the distance between the red line and black line in Figure 4(a) is closer than that in Figure 4(b). We leverage R^2 scores [64] to further quantify the superiority of using normalized data. R^2 score is a goodness-of-fit measure for linear models that indicates the extent to which the variance of one variable explains the variance of the second variable and shows how well the regression model explains the data. Table 2 displays the R^2 scores calculated with the true data and the estimated data from the 15 real-world targets used in our preliminary study (i.e., the **Norm. R^2** and **Orig. R^2** columns) and the scores larger than 0.8 are marked in bold. An R^2 score that is larger than 0.8 is typically interpreted as that over 80% of the variability observed in the target variable is explained by the model [8]. With Table 2 we can observe that: (1) In nine out of the 15 studied fuzz targets, the R^2 scores computed with the normalized data are larger than that of the original data, which indicates that the normalized data can always fit the model $y = \mu x$ better than the original data in practice; (2) In more than half of the studied targets (six out of 15), the R^2 scores achieved by the normalized

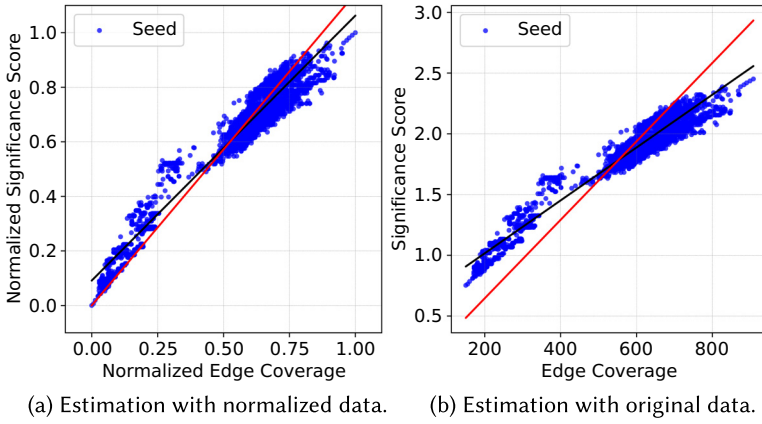


Fig. 4. Visualization of the estimations of (normalized) significance scores using data generated by fuzzing the real-world target objdump for 24 hours. The black lines represent the estimation with the OLS estimators, while the red lines represent the estimation with the model $y = \mu x$.

data are larger than 0.8, which reveals that the model $y = \mu x$ can explain the normalized data well in the vast majority of the cases.

4.3.2 Estimation-Based Power Scheduling. Algorithm 3 displays the pseudo-code of the estimation-based power scheduling. We collectively denote the global variables used by FUNFUZZ as “FUNFUZZ Globals” (line 2), which includes the maximum, minimum, and mean values of the significance scores and the coverage generated achieved since fuzzing started. At the beginning of the estimation-based power scheduling, FUNFUZZ calculates the estimation coefficient μ with *FG*. According to Section 4.3.1, μ is calculated as the ratio of the mean normalized significance scores and the mean normalized coverage when normalized data are used for estimation. Suppose the min-max normalization is used, for any dataset $X = \{x_1, x_2, \dots, x_n\} (n \in \mathbb{N}^*)$ where the normalized mean value is $\tilde{\bar{x}}$ and the mean normalized value is $\tilde{\bar{x}}$, the following equation is always established:

$$\tilde{\bar{x}} = \frac{\frac{1}{n} \sum_{i=1}^n x_i - \min(X)}{\max(X) - \min(X)} = \frac{1}{n} \sum_{i=1}^n \frac{x_i - \min(X)}{\max(X) - \min(X)} = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i = \tilde{\bar{x}}, \quad (10)$$

Let $\tilde{\bar{sig}}$ denote the normalized mean significance score and $\tilde{\bar{cov}}$ the normalized mean coverage, with Equation (10), we can calculate the estimation coefficient μ during fuzzing as follows:

$$\mu = \frac{\tilde{\bar{sig}}}{\tilde{\bar{cov}}} = \frac{\bar{sig} - sig_{min}}{sig_{max} - sig_{min}} \div \frac{\bar{cov} - cov_{min}}{cov_{max} - cov_{min}}, \quad (11)$$

where the minimum and maximum values of significance scores and coverage are subscripted with *min* and *max*. After determining the estimation coefficient, FUNFUZZ computes the scale factor ω with the real significance score *sig* and the estimated significance score $\hat{sig}$ (lines 5–7). Let γ denote the estimation error that calculated as $\gamma = sig/\hat{sig} - 1$, the scale factor ω is calculated as follows:

$$\omega = \begin{cases} \omega_{max} & \text{if } 1 + \rho * \gamma > \omega_{max} \\ \omega_{min} & \text{if } 1 + \rho * \gamma < \omega_{min} \text{ or } \hat{sig} = 0 \\ \omega = 1 + \rho * \gamma & \text{otherwise,} \end{cases} \quad (12)$$

Algorithm 3: Estimation-based Power Scheduling

```

1: global variables
2:   FUNFUZZ Globals  $FG$ 
3: end global variables
Input: Seed  $s$ 
Output: Assigned Energy  $e$ 
4:  $\mu \leftarrow \text{COMPUTEESTCOEFFICIENT}(FG)$ 
5:  $\hat{sig}, \hat{cov} \leftarrow \text{COMPUTENORMVALUES}(s, FG)$   $\triangleright$  Calculating scale factor
6:  $\hat{sig} \leftarrow \mu * \hat{cov}$ 
7:  $\omega \leftarrow \text{COMPUTESCALEFACTOR}(sig, \hat{sig})$ 
8:  $e \leftarrow \text{BASEPOWERSCHEDULING}(s)$   $\triangleright$  Adjusting assigned energy
9:  $e \leftarrow e * \omega$ 
10: return  $e$ 

```

where ω_{max} , ω_{min} and ρ are hyperparameters that are set to 2.0, 0.2, and 2.0 by default. FUNFUZZ adjusts the assigned energy by multiplying the basic energy assigned by other scheduling strategies (e.g., AFL++ default scheduling) with the calculated scale factor ω (lines 8–9). The adjusted energy is eventually returned and will be later used for scheduling input generation.

5 Implementation

We prototype FUNFUZZ by extending on AFL++ (version 4.04c). As mentioned in the first paragraph of Section 4, FUNFUZZ is composed of four components, namely the metadata extractor, the instrumentor, the fuzzer, and the FS calculator. We next elaborate on these components in sequence.

5.1 Metadata Extractor

We implement the metadata extractor as three sub-components. The first sub-component is for extracting raw program metadata from the PUT, and the other two are for further processing. Specifically, we implement the first sub-component as an LLVM pass [45], which traverses the PUT at the instruction level to collect function names and call relations and write them to two files. As for the rest two sub-components, we implement them as Python scripts. Specifically, the second sub-component takes the function name file as input. It assigns unique IDs for each recorded function name and writes the one-to-one mappings from function names to IDs into a file. The third sub-component takes the call relation file and the mapping file as input to build and store the call graph for the PUT. Note that we also implement a static mode, which calculates FS values with static call probabilities parsed from stored call relations, to investigate the effectiveness of using dynamic CC. All of the graph manipulations are implemented with a well-examined third-party library named `networkit` [57] that was previously adopted by prior work [72].

5.2 Instrumentor

We implement the instrumentor on top of AFL++’s default instrumentation module. In addition to instructions instrumented by AFL++, we emit instrumentation at (1) the front of direct calls to record CC by increasing corresponding counters by 1 and (2) the first reachable instructions of functions to update functions’ execution statuses by setting the flag to 1. The recorded CC and function statuses are passed to the fuzzer via shared memories at the fuzzing stage.

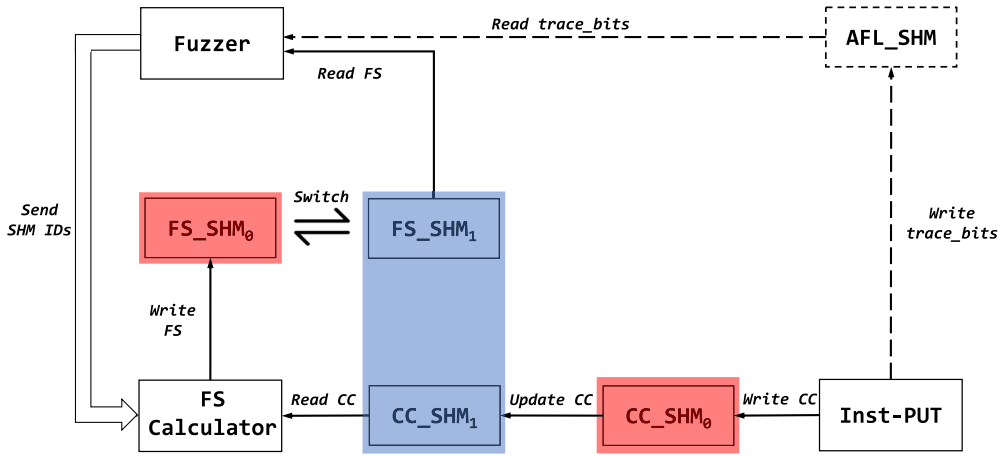


Fig. 5. The framework of the lazy FS values computation. The cold SHMs (for reading) are masked with blue and the hot SHMs (for writing) are masked with red. The dotted lines represent components and operations of AFL++, while the solid lines represent those of FunFuzz. The hollow arrow between the *Fuzzer* and the *FS Calculator* represents the pipe used for sending the request that contains the SHM IDs.

5.3 Fuzzer

The fuzzer component of FunFuzz is piggybacked on AFL++’s fuzzing module. It runs the estimation-based power scheduling (described in Section 4.3) and comprises two key designs (i.e., the **double-shared memories (SHMs)** infrastructure and the lazy FS computation). We next elaborate on the two designs in order.

5.3.1 Double-SHMs Infrastructure. FunFuzz relies on a double-SHMs infrastructure to alleviate the computation overhead brought by dynamically updating FS. Before fuzzing starts, the fuzzer first initializes the environment needed for FunFuzz and chooses the mode of using FS values. In static mode, the fuzzer directly reads the FS values prepared by the metadata extractor from the local file and uses them for fuzzing. Whereas in dynamic mode, the fuzzer first sets up a pipe to the FS calculator ahead of the main fuzzing loop and then periodically requests the FS calculator to update the FS values with fresh CC during the loop. Figure 5 displays the architecture of the double-SHMs infrastructure, which is the foundation of the lazy FS computation. Specifically, we prepare two areas of SHM for both the CC and FS values, one SHM for reading, and one SHM for writing. We refer to the SHMs for writing as “hot SHMs,” as the data preserved in these SHMs are volatile at runtime, and the SHMs for reading as “cold SHMs,” as the data preserved in these SHMs are steady. The first batch of FS values is computed with the CC generated via dry run (i.e., runs on the fuzz target with the initial seeds). These FS values are written to both FS SHMs to avoid reading from a cold FS SHM after the first switch.

5.3.2 Lazy FS Computation. The fuzzer checks the time elapsed since the last update of FS values at the beginning of each iteration of the fuzzing loop. The fuzzer conducts the following steps once the elapsed time has exceeded the preset fun period: (1) Reading CC from the hot CC SHM and overriding the cold CC SHM; (2) Emptying the hot CC SHM; (3) Switching cold and hot FS SHMs; (4) Emptying current hot FS SHM; (5) Sending the IDs of the current hot FS SHM and cold CC SHM to the FS calculator. With the lazy strategy, the fuzzing loop and the computation of FS values can be promoted in parallel to guarantee the efficiency of the whole process. We use the 1st to n th elements of the hot CC SHM to store the execution statuses of functions, where n is

the number of functions under test. We refer to the n elements as function flags that take value in $\{0, 1\}$, where 0 for not executed functions and 1 for executed ones. The fuzzer resets all the function flags to 0 before each run of the fuzz target. Before adding an interesting input to the seed queue, the fuzzer computes the significance score for the test input according to Equation (7). According to the significance scores, the fuzzer performs the estimation-based power scheduling as described in Section 4.3. The fuzzer stops the fuzzing loop when it is terminated or the time limit is reached.

5.4 FS Calculator

We implement the FS calculator in two parts with Python. The first part is for initialization. The FS calculator first reads the function map and call graph files created by the metadata extractor to prepare the call graph skeleton. The prepared graph skeleton is then utilized to initialize an instance for the Katz algorithm. FS calculator eventually builds a two-dimensional to preserve global CC after all setups for influence analysis are done. The second part is for conducting influence analysis, which is implemented as an endless loop. At the beginning of the loop, the FS calculator constantly waits for the request sent from the fuzzer; it promotes the computation of FS values once a request message comes. To update FS values, the FS calculator parses the request message to obtain SHM IDs, reads CC from the cold CC SHM, and computes fresh FS values according to Algorithm 2. Note that we remove self-circles from the graph before conducting influence analysis as they can make Katz algorithm fail to converge [56]. The updated FS values are finally written to the hot FS SHM and wait for the fuzzer to read.

6 Evaluation

In this section, we first describe the experimental setup and then present the experimental results. To evaluate FUNFUZZ, we focus on the following research questions:

- RQ1: How effective FUNFUZZ is in covering code?
- RQ2: How effective FUNFUZZ is in exposing crashes?
- RQ3: Does FUNFUZZ perform better when using dynamic FS values?
- RQ4: How does the FS-based estimation compare with the edge-only estimation?
- RQ5: Whether different fun periods influence the throughput of FUNFUZZ?

6.1 Experimental Setup

6.1.1 Fuzz Targets and Initial Seeds. We conduct experiments on a benchmark containing 15 fuzz targets, where 10 are from real-world projects widely adopted for evaluating fuzzers [10, 80, 83] and five are from Magma, a standard fuzzer evaluation benchmark injected with real vulnerabilities [27, 30]. Among the targets from real-world projects, nine of them are from the FairFuzz benchmark [38], whereas the remaining one (i.e., mjs) is from our motivating example. Table 3 displays the details of the fuzz targets, where the upper 10 rows show the real-world targets while the lower five rows show the Magma targets. The names and versions of the projects are shown in the “Project” column. The “Target & Cmdline” column displays the names of the binary programs and the command line arguments used for execution. We follow Lemieux and Sen [38] to prepare initial seeds. Specifically, for fuzz target `cxxfilt` that requires a string as input, we prepare “_Z1fv” as the initial seed; for other fuzz targets that require input files of certain formats, we pick one file for each from AFL test cases [1]. The “Initial Seeds” column displays the names of the AFL test cases selected as initial seeds. Note that the initial seeds for the nine targets from the FairFuzz benchmark conform to the original setup of FairFuzz evaluation, whereas the initial seed of `mjs` is prepared by imitating FairFuzz’s setup. As for Magma, it integrates more than 20 popular fuzzers and benchmarks 24 targets from seven different projects. To conduct experiments on Magma, we

Table 3. Summary of the Studied Fuzz Targets

	TID	Project	Target & Cmdline	Initial Seeds
<i>Real-world</i>	T01	binutils-2.28	cxxfilt	"_Z1fv"
	T02		nm-new	small_exec.elf
	T03		objdump -d	
	T04		readelf -a	
	T05	libjpeg-turbo-1.5.1	djpeg	not_kitty.jpg
	T06	libpng-1.6.29	readpng	not_kitty.png
	T07	mjs-2.20	mjs -f	small_script.js
	T08	mupdf-1.9	mutool draw	small.pdf
	T09	tcpdump-4.9.0	tcpdump	small_capture.pcap
	T10	libxml2-2.9.4	xmllint	small_document.xml
<i>Magma</i>	T11	libpng	libpng_read_fuzzer	Magma Default [48]
	T12	libsndfile	sndfile_fuzzer	
	T13	libxml2	libxml2_xml_read_memory_fuzzer	
	T14	lua	lua	
	T15	sqlite3	sqlite3_fuzz	

first add supports for two additional fuzzers, namely FUNFUZZ and AFLGo, and then try to build the targets with all the studied fuzzers. The builds of Magma targets include multiple steps, including fetching project source code, installing dependencies, conducting fuzzer-specific instrumentation on the fetched source code, etc. After several runs of attempts, we finally succeeded in building the five targets listed in Table 3 for all the studied fuzzers and thus adopted them for experiments.

6.1.2 Fuzzers. Our experiment includes eight fuzzers: FUNFUZZ-Dynamic, FUNFUZZ-Static, FUNFUZZ-EdgeOnly, AFL++, FairFuzz, AFLGo-AllFunc, Entropic, and K-Scheduler. Specifically, FUNFUZZ-Dynamic and FUNFUZZ-Static are three configurations of FUNFUZZ, where FUNFUZZ-Dynamic takes dynamic FS values computed at runtime, FUNFUZZ-Static takes static FS values computed according to the static call relations extracted at the preparation stage, and FUNFUZZ-EdgeOnly is a shadow mode that performs estimation-based scheduling with achieved edges. For simplicity, we use FUNFUZZ to stand for FUNFUZZ-Dynamic as it is the default configuration. AFL++ is the superior fork of the well-known greybox fuzzer AFL [23, 82]. It supports all the features of AFL and integrates novel ideas from the fuzzing community. We choose AFL++ as it is the base of FUNFUZZ. FairFuzz is a fuzzer with specific focuses on rarely executed branches [38]. We choose FairFuzz as it also leverages dynamic call probabilities. Note that AFLFast is also a greybox fuzzer using dynamic program information for power scheduling [11]; it has been integrated into current AFL++ as the default power scheduling strategy (version 4.04c) and thus a comparison to AFL++ naturally enable the comparison to it [2]. AFLGo is the seminal directed greybox fuzzer that aims at reaching given program sites [10]. The "AllFunc" denotes a special configuration that specifies all the functions under test as target sites. We include AFLGo-AllFunc as it is conceptually associated with FS and can potentially achieve a similar effect. To realize "AllFunc," we implement (1) an LLVM pass to extract the names and definition line numbers of the functions under test and (2) a Python script to transform the extracted function information into the form of target sites required AFLGo. The modified version of AFLGo can be found in our artifacts [24]. For clarity, we use AFLGo to denote AFLGo-AllFunc in the rest of the section. Entropic is a SOTA greybox fuzzer that adopts innovative guidance based on information theory [9]. We include Entropic as it is a representative Libfuzzer-based fuzzer and also leverages runtime information [43]. K-Scheduler is another SOTA greybox fuzzer equipped with an effective seed scheduling technique powered

by Katz centrality [72]. We include K-Scheduler because it employs the same algorithm (i.e., Katz centrality) with current FUNFUZZ implementation. For brevity, we shorten K-Scheduler as “K-Sched” when discussing experimental results.

6.1.3 Fuzz Campaigns and Infrastructure. A fuzz campaign is a specific execution of fuzzing regarding a certain fuzz target for a given duration [50]. We prepare 85 fuzz campaign setups and set the duration to 24 hours following pioneer works [12, 33]. Specifically, we run the 10 real-world targets with five fuzzers (i.e., $10 \times 5 = 50$ campaigns) and run the five targets with six fuzzers (i.e., $5 \times 6 = 30$ campaigns). The five fuzzers run on the real-world targets are FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo; whereas the six fuzzers run on the Magma targets are FUNFUZZ, AFL++, FairFuzz, AFLGo, Entropic, and K-Sched. Besides the 80 campaigns for comparing fuzzers’ performance, we prepare five campaigns to run FUNFUZZ-EdgeOnly and evaluate the contribution of the FS-based estimation scheduling. We repeat each campaign for 20 trials to generate plenty of raw data and alleviate randomness. The fuzz campaigns on the real-world targets are performed on a machine with 16 cores (Intel®; Xeon®; Silver 4114 CPU @ 2.20GHz) and 32 GB memory running Ubuntu 18.04 operating system, while the campaigns on the Magma targets are performed on a machine with 32 cores (13th Gen Intel(R) Core(TM) i9-13900K) and 32 GB memory running Ubuntu 22.04 operating system.

6.2 RQ1: The Performance on Covering Code

We leverage LLVM region coverage to measure the capabilities of the fuzzers in covering code [17]. We choose LLVM region coverage as it is (1) standard and thus can enable the coverage comparison among fuzzers from different families (e.g., the AFL-based FairFuzz and the LibFuzzer-based Entropic) and (2) widely adopted in fuzzing evaluation [12, 53]. For targets from real-world projects (i.e., T01–T10 in Table 3), we re-compile them with LLVM coverage options, reproduce the test inputs generated by the studied fuzzers, and leverage LLVM coverage tools (e.g., `llvm-cov`) to collect region coverage. For targets from Magma (i.e., T11–T15 in Table 3), we leverage the builtin `llvm_cov` fuzzer [49], which equips with a fuzz driver tailored for reproducing cases, to collect region coverage. Note that the rationale of the reproduction with `llvm_cov` is essentially similar to our manual LLVM-based reproduction of the real-world targets. The coverage results of the real-world targets are discussed in Section 6.2.1, while the results of the Magma are discussed in Section 6.2.2.

6.2.1 Coverage Results of the Real-World Targets. Table 4 displays the region coverage results achieved by FUNFUZZ, AFL++, FUNFUZZ-Static, FairFuzz, and AFLGo on the 10 real-world targets, where the “#Reg.” columns show the absolute number of covered regions averaged over 20 trials and the “ Δ (%)” columns show the increase/decrease percentages of covered regions relative to AFL++. The results that get relative increases of more than 1.0% are shaded in gray, meaning that the studied fuzzers achieve relatively significant improvements in code coverage on the corresponding targets compared to AFL++. As shown in Table 4, FUNFUZZ outperforms AFL++ on all 10 targets by improving 0.1%–18.4% region coverage, where the increase amounts are larger than 2.0% on eight of the targets; it achieves the best average region coverage on eight targets except for `tcpdump` and `xmllint`, where FairFuzz shows to be the best and FUNFUZZ is close behind. Although FUNFUZZ-Static does not perform as well as FUNFUZZ, it also outperforms AFL++ by achieving 0.1%–7.4% more region coverage on eight of the targets. The results demonstrate that FUNFUZZ can effectively facilitate the host fuzzer (i.e., AFL++ in our evaluation) in terms of code coverage. The facilitation is more significant under the dynamic mode (i.e., the default configuration) than the static mode (i.e., the FUNFUZZ-Static configuration).

Table 4. Summary of the Final LLVM Region Coverage Achieved by AFL++, FUNFUZZ, FUNFUZZ-Static, FairFuzz, and AFLGo on Real-World Targets (i.e., T01–T10)

TID	Target	AFL++	FUNFUZZ		FUNFUZZ-STATIC		FairFuzz		AFLGo	
		#Reg.	#Reg.	Δ (%)	#Reg.	Δ (%)	#Reg.	Δ (%)	#Reg.	Δ (%)
T01	cxxfilt	4,546.2	4,557.2	0.2	4,545.5	0.0	4,362.1	−4.1	4,163.1	−8.4
T02	nm-new	5,609.1	5,731.7	2.2	5,632.8	0.4	5,480.3	−2.3	5,265.3	−6.1
T03	objdump	8,297.1	9,080.2	9.4	8,400.5	1.3	7,631.1	−8.0	8,751.9	5.5
T04	readelf	14,333.1	14,959.0	4.4	14,342.4	0.1	14,535.1	1.4	14,603.9	1.9
T05	djpeg	3,441.5	3,611.5	4.9	3,496.1	1.6	3,431.5	−0.3	3,271.8	−4.9
T06	readpng	1,617.7	1,685.4	4.2	1,650.2	2.0	1,634.7	1.1	1,611.3	−0.4
T07	mjs	2,709.6	2,824.5	4.2	2,763.2	2.0	2,524.4	−6.8	2,574.1	−5.0
T08	mutool	3,876.8	3,881.2	0.1	3,877.2	0.0	3,865.3	−0.3	—	—
T09	tcpdump	8,385.9	9,926.7	18.4	9,002.4	7.4	10,160.6	21.2	7,265.6	−13.4
T10	xmllint	17,482.0	18,610.6	6.5	17,898.4	2.4	18,616.2	6.5	15,071.7	−13.8

The “#Reg.” columns are the average region coverage over 20 trials of fuzz campaigns and the “ Δ (%)” are the coverage percentage relative to AFL++.

To further profile how the region coverage increases with the proceeding of fuzzing, we depict trends with region coverage sampled every 15 minutes. Figure 6 presents the tendencies of region coverage among 24-hour fuzz campaigns, where the results achieved by the five fuzzers on the 10 real-world targets are shown in Figure 6(b)–(j). The curves of FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo are colored in red, magenta, blue, green, and yellow. The light-colored areas visualize the ranges of edge coverage of the eight fuzzers, where the upper/lower bound represents the biggest/smallest region coverage sample at the corresponding time points. The lines centered in these areas are drawn using average coverage. According to Figure 6, the red line of FUNFUZZ is above or overlapped with the other four studied fuzzers in almost all the 15 targets, indicating FUNFUZZ’s superiority on code coverage. The only exceptions are xmllint, tcpdump, where FairFuzz shows the best. FUNFUZZ-Static shows a slightly better performance than AFL++ and is remarkably inferior to FUNFUZZ on objdump, readelf, djpeg, readpng and xmllint. This result further demonstrates the effectiveness of FUNFUZZ and the superiority of dynamic FS computation.

6.2.2 Coverage Results of the Magma Targets. Table 5 shows the region coverage results achieved by AFL++, FUNFUZZ, FairFuzz, AFLGo, Entropic, and K-Sched on the five Magma targets. The structure of Table 5 conforms to that of Table 4 except that the target names are omitted due to space limits. According to Table 5, FUNFUZZ outperforms AFL++ on *T13*–*T15*, but underperforms AFL++ on the rest two targets (i.e., *T11* and *T12*). The other four fuzzers are far behind FUNFUZZ and AFL++ regarding region coverage on almost all the five targets except for Entropic on *T11*. The results that FUNFUZZ outperforms SOTA fuzzers like Entropic and K-Sched demonstrate the effectiveness of FUNFUZZ in covering code. Delving into the results of AFL++ and FUNFUZZ, we find that *T13*–*T15* are all targets that require structured inputs (i.e., XML, LUA, and SQL), while the input formats of the rest two targets are less structured (i.e., PNG and audio files). Associating that Magma provides richer initial seed copra for the benchmarked targets, these experimental results imply that FUNFUZZ is better at tackling targets that require complex-structured inputs when plenty of initial seeds are given. Figure 6(k)–(o) illustrate the coverage trends of the six studied fuzzers on Magma targets, where the results of FUNFUZZ, AFL++, FairFuzz, AFLGo, Entropic, and K-Sched are

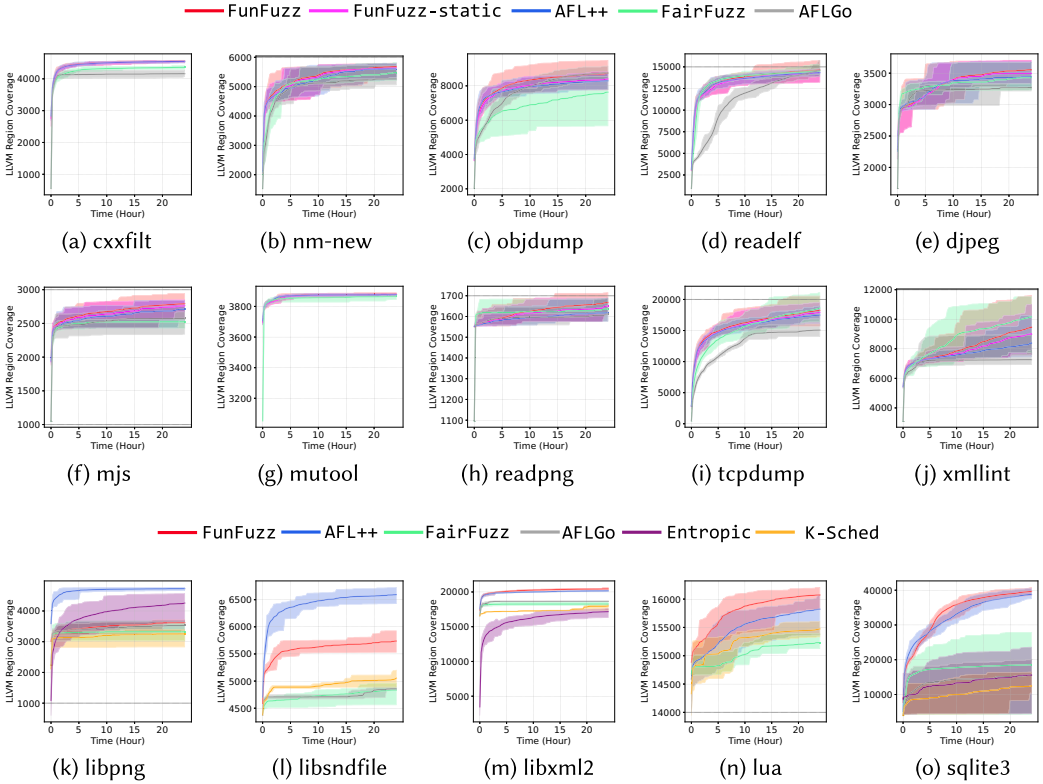


Fig. 6. Tendencies of LLVM region coverage achieved by the seven studied fuzzers on real-world and Magma targets, where(b)-(j) are curves of real-world targets and (k)-(o) are curves of Magma targets. The X-axis displays the execution time in hours and the Y-axis displays the number of regions. The solid lines represent the coverage averaged over 20 trials whereas the light-colored region depicts the range of coverage.

Table 5. Summary of the Final LLVM Region Coverage Achieved by AFL++, FunFuzz, AFLGo, Entropic, and K-Sched on Magma Targets (i.e., T11–T15)

TID	AFL++	FunFuzz		FairFuzz		AFLGo		Entropic		K-Sched	
	#Reg.	#Reg.	Δ (%)	#Reg.	Δ (%)	#Reg.	Δ (%)	#Reg.	Δ (%)	#Reg.	Δ (%)
T11	4726.4	3621.8	−23.4	3323.3	−29.7	3530.0	−25.3	4258.5	−9.9	3253.5	−31.2
T12	6594.2	5737.7	−13.0	4833.2	−26.7	4857.8	−26.3	—	—	5054.2	−23.4
T13	20,166.5	20,445.2	1.4	18,268.6	−9.4	18,646.5	−7.5	17,168.0	−14.9	17,984.8	−10.8
T14	15,821.6	16,072.7	1.6	15,231.1	−3.7	—	—	—	—	15,474.4	−2.2
T15	38,785.7	39,635.2	2.2	18,535.5	−52.2	15,599.0	−59.8	15,557.8	−59.9	12,436.8	−67.9

The “#Reg.” columns are the average region coverage over 20 trials of fuzz campaigns and the “ Δ (%)” are the coverage percentage relative to AFL++.

colored in red, blue, green, gray, purple, and orange. As the curves illustrate, FunFuzz performs the best on *T13–T15* and finishes third and second on *T11* and *T12*. The results further demonstrate that FunFuzz is competitive with SOTAs and can better handle targets requiring structured inputs.

```

==30332==ERROR: AddressSanitizer: global-buffer-overflow on address ...
READ of size 8 at ... thread T0
#0 0x5b7bc6 in OP_E_register .../binutils-2.28/opcodes/i386-dis.c:14994:12
#1 0x5adbdb7 in print_insn .../binutils-2.28/opcodes/i386-dis.c:13250:3
#2 0x4fbac2 in disassemble_bytes .../binutils-2.28/binutils/.objdump.c:1801:17
#3 0x4fbac2 in disassemble_section .../binutils-2.28/binutils/.objdump.c:2241:7
.....(more stacks)

0x000000ca26a8 is located 24 bytes to the left of global variable 'att_names_xmm' defined in
'i386-dis.c:3309:20' (0xca26c0) of size 256
0x000000ca26a8 is located 8 bytes to the right of global variable 'att_names_bnd' defined in
'i386-dis.c:3294:20' (0xca2680) of size 32
SUMMARY: AddressSanitizer: global-buffer-overflow .../binutils-2.28/opcodes/i386-dis.c:14994:12
in OP_E_register
.....(more crash details)

```

Fig. 7. A piece of crash frame intercepted from a crash report of objdump.

6.3 RQ2: The Performance on Finding Bugs

Our evaluation of bug-finding capabilities is composed of two parts. First, for targets from real-world projects (i.e., T01–T10 in Table 3), we reproduce the generated test inputs with **AddressSanitizer (ASAN)** instrumented targets and leverage the exposed crashes as the indicator. Second, for targets from Magma (i.e., T11–T15 in Table 3), we leverage the benchmark tools provided by Magma developers to extract triggered bugs and then conduct further analyses. The methodology of identifying unique crashes is described in Section 6.3.1. The detailed statistics of exposed crashes and triggered Magma bugs are presented in Sections 6.3.2 and 6.3.3. We also discuss the overlaps of the exposed unique crashes and triggered Magma bugs in Section 6.3.4.

6.3.1 Identifying Unique Crashes. To identify the crashes found by the fuzzers, we compile the fuzz targets with ASAN to instrument instructions for unveiling memory bugs [16, 72]. Next, we rerun fuzzer-generated test cases against ASAN targets to collect raw crash reports. After that, we analyze ASAN crash frames and deduplicate the recorded crashes in a semi-automatic way. Specifically, we first traverse the crash logs output by reproducing the fuzzing-generated test cases to understand the characteristics of crash frames and then write Python scripts to uniquely mark crashes by combining the first crash locations and the crash type provided in the summary sections. Figure 7 exemplifies a piece of the crash frame intercepted from objdump. Commonly, an ASAN-style crash frame consists of four parts: (1) a header with error messages, (2) lots of crash stacks with error address and code locations, (3) crash summaries with the type of crash (global-buffer-overflow as exemplified), and (4) detailed analyses of the crash. For the crash frame shown in Figure 7, we uniquely identify it as “{i386-dis.c:14994, global-buffer-overflow}”.

6.3.2 Statistics of Exposed Unique Crashes. Table 6 summaries the crashes exposed by the five studied fuzzers on seven fuzz targets, namely cxxfilt, nm-new, objdump, readelf, tcpdump, xmllint, and mjs. The rest three fuzz targets are excluded as no crash is reported by any fuzzers during our reproduction. In Table 6, the # columns display the total number of crashes exposed by each fuzzer on certain targets, and the *Details* columns exhibit the numbers and types of specific crashes. The ML, SO, ASTB, etc. listed in the *Details* cells are abbreviated mnemonics for crash types like “memory-leak,” “stack-overflow,” and “allocation-size-too-big.” The *Sum* row displays the total numbers of crashes or types achieved by certain fuzzers, where the crash numbers are listed in # cells and the type numbers are listed in *Details* cells. As shown in Table 6, FUNFUZZ exposes the most unique crashes (114 in *Sum*), whereas AFLGo exposes the least (50 in *Sum*). In terms of the types of exposed crashes, FairFuzz performs the best among the five studied fuzzers (25 in *Sum*), whereas FUNFUZZ comes into the second place (23 in *Sum*). However, although FUNFUZZ finds slightly fewer

Table 6. Summary of Crashes Exposed by the Eight Studied Fuzzers on Seven Fuzz Targets

Fuz. Tar.	AFL++		FairFuzz		FUNFUZZ		FUNFUZZ-STATIC		AFLGo	
	#	Details	#	Details	#	Details	#	Details	#	Details
cxxfilt	32	ML 1 SO 30 ASTB 1	10	ML 1 SO 8 ASTB 1	31	ML 1 SO 29 ASTB 1	25	ML 1 SO 23 ASTB 1	17	ML 1 SO 15 ASTB 1
nm-new	5	ML 1 OOM 2 ASTB 2	7	ML 1 OOM 3 HBO 1 SBO 1 ASTB 1	5	ML 1 OOM 2 ASTB 2	5	ML 1 OOM 2 ASTB 2	4	ML 1 OOM 2 ASTB 1
objdump	7	OOM 2 ASTB 3 GBO 2	5	OOM 1 HBO 1 ASTB 1 GBO 2	9	OOM 3 SBO 1 ASTB 3 GBO 2	8	OOM 3 ASTB 3 GBO 2	5	OOM 1 HBO 1 ASTB 1 GBO 2
readelf	6	ML 2 OOM 1 HBO 3	12	ML 2 OOM 1 HBO 6 SEGV 3	13	ML 2 OOM 1 HBO 7 SEGV 3	7	ML 2 OOM 1 HBO 4	5	ML 2 OOM 1 HBO 2
tcpdump	5	HBO 5	12	HBO 12	8	OOM 1 HBO 7	10	HBO 10	15	HBO 15
xmllint	1	ML 1	2	ML 2	3	ML 3	1	ML 1	1	ML 1
mjs	37	HBO 2 SO 3 ASTB 1 SEGV 29 MPO 2	27	OOM 1 HBO 1 SO 2 ASTB 1 SEGV 19 MPO 2 NSP 1	45	ML 1 HBO 1 SO 3 ASTB 1 SEGV 38 MPO 1	30	ML 1 HBO 2 SO 5 ASTB 1 SEGV 21	3	SEGV 3
Sum	93	19	75	25	114	23	86	19	50	16

The “#” columns display the total number of crashes exposed by the fuzzers on certain targets. The “Details” columns show the number of crashes belonging to different types. The types of crashes are presented in abbreviation. Specifically, ML stands for “memory-leak,” SO for “stack-overflow,” ASTB for “allocation-size-too-big,” HBO for “heap-buffer-overflow,” SBO for “stack-buffer-overflow,” GBO for “global-buffer-flow,” OOM for “out-of-memory,” SEGV for “segmentation fault,” MPO for “memcpy-param-overlap,” and NSP for “negative-size-param.” The “Sum” row displays the total number of crashes exposed by the studied fuzzers on all seven targets.

types of crashes than FairFuzz does, there are types of crashes that can only exposed by FairFuzz or FUNFUZZ, which implies that each of these two fuzzers has its own strengths in exposing crashes. Take mjs as an example, FairFuzz exposes one OOM crash and one NSP crash that are missed by the other three fuzzers, while also failing to expose the ML crash found by FUNFUZZ. We will further delve into the overlaps of exposed crashes in Section 6.3.4. FUNFUZZ outperforms AFL++ regarding not only the number of exposed unique crashes (114 vs. 93 in *Sum*) but also the number of types (23 vs. 19 in *Sum*), while FUNFUZZ-Static performs relatively worse than AFL++ with a slightly smaller number of exposed crashes (86 vs. 93 in *Sum*). In terms of crash types, FUNFUZZ-Static also exposes an ML crash on mjs like FUNFUZZ does, but fails to trigger the HPO crashes found by both AFL++ and FUNFUZZ. The results imply that the proposed FS-centered estimation power scheduling can assist greybox fuzzing in exposing bugs, especially when dynamic call probabilities are used for FS calculation. We will further discuss the advantages of using dynamic FS in Section 6.4.

Table 7. Statistics of Magma Bugs

Target	FunFuzz		AFL++		FairFuzz		AFLGo		Entropic		K-Sched	
	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.
T11	40	3	24	4	27	2	36	3	60	3	27	2
T12	140	7	140	7	35	7	43	3	—	—	59	6
T13	83	6	85	6	21	2	20	1	38	2	20	1
T14	20	1	19	1	19	1	—	—	—	—	20	1
T15	92	8	79	7	35	4	20	2	28	3	3	2
Sum	375	25	347	25	137	16	119	9	126	8	129	12

The “Tot.” columns show the total numbers of bugs triggered in 20 trials on the five Magma targets and the “Uni.” columns exhibit the numbers of unique triggered bugs.

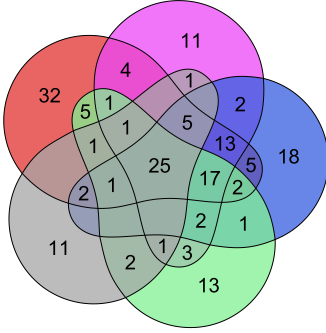
6.3.3 Statistics of Triggered Magma Bugs. The Magma developers document the types of bugs, which are also real crashes, injected in the benchmarked targets on Magma official website [30]. They also provide tools to help extract triggered bugs from raw Magma data. Based on Magma documentations and tools, we further extract the information of the triggered bugs, minimize the duplicated bugs, and gather the statistics. Table 7 exhibits the statistics of the triggered Magma bugs, where the the *Tot.* columns show the total number of bugs triggered among all 20 trials and the *Uni.* columns show the number of unique bugs after minimization. The gray-shaded cells represent the best results among all studied fuzzers. Intuitively, the *Tot.* results showcase the steadiness of bug exposing, whereas the *Uni.* results reflect the diversity. As shown in Table 7, FUNFUZZ performs the best on three out of five Magma targets in terms of total bugs, and performs the best on four out of five Magma targets in terms of unique bugs. Moreover, FUNFUZZ reaches the best regarding both the sums of total and unique triggered bugs (i.e., the left two cells of *Sum* row). The results demonstrate that FUNFUZZ is superior in both the steadiness and diversity of finding Magma bugs.

6.3.4 Overlaps of the Exposed Unique Crashes and the Triggered Magma Bugs. We visualize the bug-finding results as Venn diagrams to better show how the crashes and Magma bugs triggered by the studied fuzzers intersect. The crashes exposed by FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo are illustrated in Figure 8(a), whereas the Magma bugs triggered FUNFUZZ, AFL++, FairFuzz, AFLGo, Entropic, and K-Sched are illustrated in Figure 8(b). The colors of the bug sets of individual fuzzers are tagged at the top of the Figure 8. The numbers are the cardinals of the sets of crashes or bugs co-discovered by several fuzzers, where the zeros, which indicate no intersects, are omitted for clarity. According to Figure 8(a), the crashes or Magma bugs triggered by the studied fuzzers are not fully overlapped. Some crashes or bugs can only be found by one or some of the fuzzers. Specifically, there are 25 unique crashes found by all five fuzzers, and 32, 11, 18, 13, and 11 crashes only found by FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo. As for Magma bugs, there are five bugs found by the six experimental fuzzers in common, and one bug that is only triggered by FUNFUZZ over the 20 trials. The results displayed in the Venn diagrams showcase FUNFUZZ’s capability of finding bugs, demonstrating that FUNFUZZ can trigger bugs supplementary to other studied fuzzers.

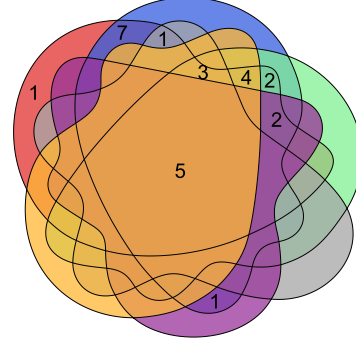
6.4 RQ3: The Benefits of Using Dynamic FS Values

In our evaluation, FUNFUZZ is the default configuration of our approach that leverages FS values computed on-the-fly, whereas FUNFUZZ-Static is a special configuration that is equipped with static FS values computed according to the static call graph (Section 5). According to Table 4, FUNFUZZ

FunFuzz FunFuzz-static AFL++ FairFuzz AFLGo Entropic K-Sched



(a) Exposed crashes.



(b) Triggered Magma bugs.

Fig. 8. Venn diagrams show the overlaps of the crashes exposed by FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo on 10 real-world targets (a) and the bugs triggered by FUNFUZZ, AFL++, FairFuzz, AFLGo, Entropic, and K-Sched on the five Magma targets (b).

```

1 void fa(char *input) {
2   if (input[0] == '!' && input[1] == '?') {
3     fb();
4   } else {
5     fc();
6   }
7 }

```

Fig. 9. A code snippet of unequal call probabilities.

outperforms FUNFUZZ-Static in all the 15 fuzz targets in terms of the final edge coverage. Moreover, according to Figure 6, the line of FUNFUZZ appears more steer than the one of FUNFUZZ-Static on five out of 15 fuzz targets. These results indicate that dynamic FS outperforms static FS in both improving fuzzing's effectiveness and efficiency in covering code. According to Table 6, FUNFUZZ exposes more unique crashes than FUNFUZZ-Static does in total. It also hits more crashes than FUNFUZZ does on five out of the seven crashing targets. These results on unique crashes demonstrate that dynamic FS is more helpful than static FS in exposing crashes.

We attribute the inferiority of using static FS values to the over-approximation nature of the static analysis [73, 75], which originates from analyzing the target program without actually running it. Although static call relations can profile the call probabilities between functions to a certain extent, they are still inadequate in many cases. Figure 9 illustrates such cases with a small piece of code. Once the function `fa` is called with a string parameter `input`, it conditionally calls the other two callees (i.e., `fb` and `fc`) according to the first and second character of `input`. When static analysis is conducted, the call relation $\langle fa, fb \rangle$ and $\langle fa, fc \rangle$ are treated equally (0.5 vs. 0.5) and are attached with the same edge weights within the constructed call graph. Such manipulation confuses FUNFUZZ with wrong FS values as the probability of executing `fb` ($1/128^2$ if taking characters in the domain of ASCII code) is far smaller than that of executing `fc`. These confusions may consequently degrade the effectiveness of FUNFUZZ. Figure 10 illustrates the fluctuations of FS values regarding two functions from the fuzz target `mjs`, where Figure 10(a) displays the tendency for function `mjs_bcode_insert_off` (`f3` in Figure 2), and Figure 10(b) for `parse_block_or_stmt` (`f4` in Figure 2). The X-axis of Figure 10 represents the elapsed time and the Y-axis represents the

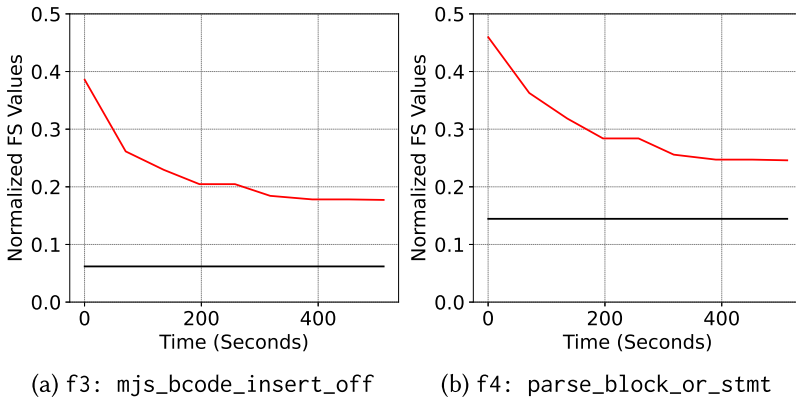


Fig. 10. The fluctuations of dynamic FS values of two functions from mjs during fuzzing.

normalized FS values. For better illustration, we normalize the recorded FS values to make sure that the static and dynamic ones can reside in the same value region and only exhibit FS values for the first 600 seconds to make the change of FS values clearer. The black lines represent the static FS values, whereas the red lines represent the dynamic FS values computed by FUNFUZZ during fuzzing. According to Figure 10, as fuzzing proceeds, the dynamic FS values of f3 and f4 gradually fall at the first 600 seconds. Being configured statically, FUNFUZZ-Static uses constant FS values and is unaware of the updates of dynamic FS values and thus may assign inappropriate energy to promising seeds.

6.5 RQ4: Comparison to Edge-Only Estimation Model

The FS-centered estimation-based power scheduling is a key mechanism of FUNFUZZ, which adjusts seed energies according to the ratios of the real and the estimated FS values. The estimated FS values are calculated by a linear model dynamically constructed during fuzzing. Given the fact that FS is somewhat positively correlated with edge (code) coverage (cf., the first question discussed in Section 4.3.1), one may naturally think that it may be feasible to conduct such estimation-based power scheduling straightforward with edge coverage and do not need to take pains on building an FS-centered model. Therefore, in this section, we investigate the effectiveness of the FS-centered estimation-based power scheduling and the necessity of building an FS-centered model by comparing FUNFUZZ to FUNFUZZ-EdgeOnly, a shadow mode that adjusts seed energies according to achieved edges, on five real-world targets. For brevity, we use EdgeOnly to denote FUNFUZZ-EdgeOnly in the rest of this section. We also include AFL++ in the experiments as it is the implementation base of both FUNFUZZ and EdgeOnly. Similar to the experiments mentioned earlier in this paper, we repeat EdgeOnly for 20 trials of 24-hour fuzzing on each target to alleviate randomness and reproduce the stored test inputs to obtain LLVM region coverage.

Figure 11 shows the trends of LLVM region coverage achieved by FUNFUZZ, AFL++, and EdgeOnly on five real-world targets, where the curves of EdgeOnly are colored in brown. As shown in Figure 11, it is evident that EdgeOnly underperforms FUNFUZZ and AFL++ on all five targets in terms of LLVM region coverage. Table 8 displays the average region coverage and the relative coverage percentage compared to AFL++, where EdgeOnly achieve decreased region coverage (i.e., -16.1% to -0.4%) on all five targets. The main reason behind the underperformance of EdgeOnly is the unawareness of the executed functions. Without FS, EdgeOnly is unable to distinguish whether coverage is attained by exercising significant functions connecting to many other functions or less significant

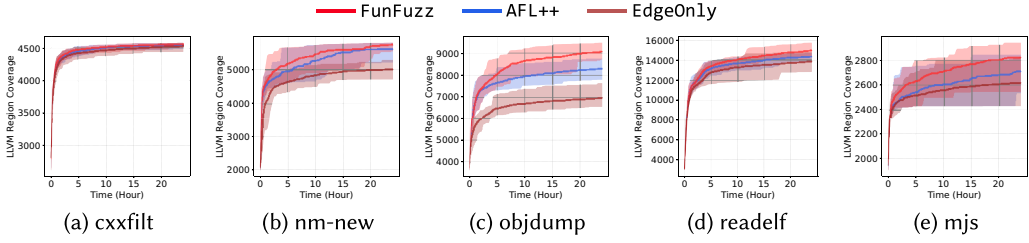


Fig. 11. Tendencies of LLVM region coverage achieved by AFL++, FUNFUZZ, and EdgeOnly on five targets.

Table 8. Summary of the Final LLVM Region Coverage Achieved by AFL++, FUNFUZZ, and EdgeOnly

TID	Target	AFL++	FUNFUZZ		EdgeOnly	
		#Reg.	#Reg.	Δ (%)	#Reg.	Δ (%)
T01	cxxfilt	4546.2	4557.2	0.2	4528.2	-0.4
T02	nm-new	5609.1	5731.7	2.2	5006.2	-10.8
T03	objdump	8297.1	9080.2	9.4	6963.7	-16.1
T04	readelf	14333.1	14959.0	4.4	13862.1	-3.3
T07	mjs	2709.6	2824.5	4.2	2617.7	-3.4

functions with low connectivity. As such, EdgeOnly may drive fuzzing into corner program regions and trap executions in a small range, consequently leading to low resultant coverage. All in all, the experimental results demonstrate the superiority of the FS-centered estimation-based power scheduling and the necessity of constructing an FS-centered estimation model.

6.6 RQ5: The Impact of the Fun Period

Throughput is critical to the effectiveness of fuzzing [26]. To prevent losing too much throughput and ensure the execution speed of FUNFUZZ, we design a lazy FS computation mechanism to update periodically FS values in an asynchronous manner Section 5. In this RQ, we evaluate the effectiveness of our lazy FS computation and investigate how different settings of the fun period influence fuzzing throughput, which is essentially the number of executions generated in concrete fuzz campaigns. We conduct the experiment by running FUNFUZZ on the target *mjs* with the fun period, which is denoted as δ in Section 4, set to ∞ , 30, 60, 90, 120, 150, and 180 seconds, where the setting “ $\delta = \infty$ ” corresponds the results of the plain AFL++. To collect raw data and alleviate randomness, we repeat each δ setting for 10 trials of 24-hour campaigns and extract the final total execution numbers recorded in the `plot_data` files. Figure 12 visualizes the variant trend of **average final total executions (AFTE)** running FUNFUZZ on *mjs* with different δ settings. The curve is basically steady with a float ranging in -2.43% to $+1.23\%$. The PCC between δ and AFTE (calculated excluding the data of the “ $\delta = \infty$ ” setting) is 0.3648, indicating that the correlation between δ and AFTE (i.e., the quantification of fuzzing throughput) is quite weak [34].

To further explore the impact of the fun period on FUNFUZZ’s throughput, we compare the distributions of the final total executions of the seven studied δ settings over all 10 trials with Mann–Whitney *U* test. The Mann–Whitney *U* test or the Wilcoxon rank sum test is a non-parametric statistic hypothesis test assessing whether there is a significant difference between two independent datasets [52]. It is widely adopted in fuzzing evaluation to check to what extent two data groups (e.g., coverage results) differ [7, 10, 12, 68, 84]. In the Mann–Whitney *U* test, if the calculated

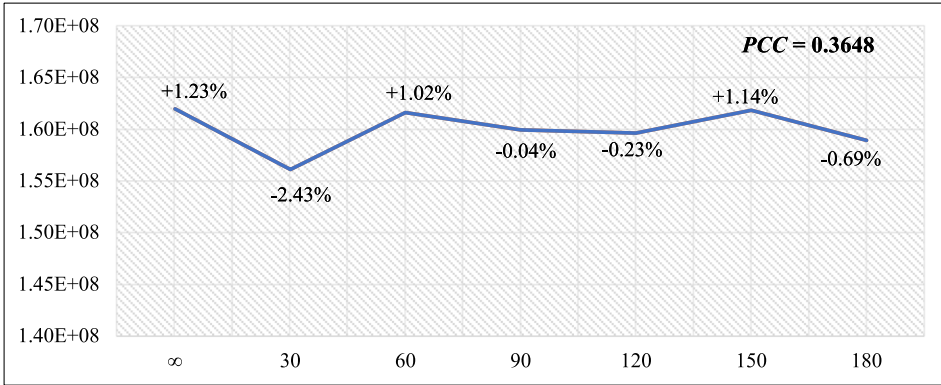


Fig. 12. Trends of AFTE running FunFuzz on mjs with different fun periods. The X-axis shows the settings of fun periods (in seconds) and the Y-axis shows the number of executions.

Table 9. The Mann–Whitney U Test p-Values among the Final Total Executions Generated by Fuzzing the Target mjs with Fun Periods Set to ∞ (i.e., AFL++), 30, 60, 90, 120, 150, and 180 Seconds

δ (Second)	∞	30	60	90	120	150	180
∞	—	0.1212	0.4727	0.3447	0.3075	0.5205	0.2730
30	0.1212	—	0.0003	0.0010	0.0036	0.0002	0.0036
60	0.4727	0.0003	—	0.0890	0.1041	0.9097	0.0113
90	0.3447	0.0010	0.0890	—	0.5708	0.0140	0.1620
120	0.3075	0.0036	0.1041	0.5708	—	0.0113	0.6776
150	0.5205	0.0002	0.9097	0.0140	0.0113	—	0.0004
180	0.2730	0.0036	0.0113	0.1620	0.6776	0.0004	—

The cells indicating “no significant difference” (i.e., p-value > 0.05) are colored in gray.

p-values of two datasets are ≤ 0.05 , then these two datasets are considered significantly different. Or else, if the resultant p-values are > 0.05 , the two datasets *cannot* be considered significantly different since the evidence is insufficient. Table 9 display the Mann–Whitney U test results between every two δ settings, where the p-values > 0.05 are highlighted in gray. As shown in Table 9, the distribution of the final total executions under the setting “ $\delta = 30$ s” is significantly different from the other settings except for “ $\delta = \infty$,” while the distributions under the other five settings are basically similar, especially “ $\delta = 60$ s,” “ $\delta = 90$ s,” and “ $\delta = 120$ s.” Associating that AFTE achieved under the setting “ $\delta = 30$ s” is the lowest (with a decline of -2.43% compared to the average AFTE) among all seven settings, the lazy FS computation mechanism seems to bring slightly more bad adversities when the fun period is shorter than 60 s. This result is unsurprising as shorter fun periods will incur more frequent updates of FS values, and a more frequent update will certainly lead to more overhead, which further supports our choice of setting “ $\delta = 60$ s” as default. Interestingly, the p-values between the data distribution of “ $\delta = \infty$ ” and all δ settings are larger than 0.05, implying that FS computation won’t influence fuzzing throughput significantly. Overall, the experimental results imply that the fun period has no significant impact on FunFuzz’s throughput, especially when set to more than 60 seconds, demonstrating the effectiveness of the lazy FS computation in ensuring fuzzing throughput.

Table 10. The Average Numbers of the Indirect Calls Covered by FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo on the 10 Real-World Targets over 20 Trials of 24-Hour Campaigns

	FUNFUZZ	FUNFUZZ-STATIC	AFL++	FairFuzz	AFLGo
cxxfilt	135.1	134.2	127.4	109.3	80.9
nm-new	326.3	324.0	321.5	314.9	308.2
objdump	508.8	487.2	486.0	456.9	497.7
readelf	267.0	262.3	261.9	235.7	222.7
djpeg	330.8	325.1	323.3	321.6	306.3
readpng	82.0	82.0	82.0	82.0	82.0
mjs	72.7	72.6	72.0	70.0	71.1
mutool	146.9	147.0	146.9	146.9	—
tcpdump	73.0	73.0	73.0	73.0	73.0
xmllint	304.4	287.4	276.7	310.6	244.0

The best results are highlighted in gray.

7 Discussion

In this section, we discuss some missing aspects of the design and evaluation of FUNFUZZ. We first investigate how well FUNFUZZ and other studied fuzzers can tackle indirect calls and then conclude the threats to validity.

7.1 Handling Indirect Calls

Indirect calls are an important way to implement runtime dispatching in modern software, which typically refers to invoking a function where the address of the callee function is unknown at compile-time, and instead, is determined at runtime [13, 46]. As for the design of FUNFUZZ, we choose to conduct FS computation upon call groups constructed with information (i.e., direct calls) extracted through compile-time analyses (Section 4). Although common in fuzzing research, such a choice may undermine techniques' capability in uncovering code as it fails to take into account indirect calls that are not determined until runtime, thereby possibly leaving out code regions hidden behind them [10, 84]. To explore how the compile-time call graphs impact the effectiveness of FUNFUZZ, we conduct further analyses of FUNFUZZ and other compared fuzzers, namely FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo, regarding indirect calls. Specifically, we leverage DynamoRIO [20], a runtime code tracing tool to reproduce the fuzzer-generated test inputs and collect indirect calls covered by the five studied fuzzers. To count unique indirect calls, we remove the execution-specific dynamic addresses within the DynamoRIO records and eliminate the duplicated records according to the source and destination of calls.

Table 10 summarizes the number of indirect calls achieved by the five studied fuzzers averaged over the 20 trials of 24-hour fuzzing, where the cells showing best results are shaded with gray. As illustrated by Table 10, FUNFUZZ performs the best on six out of the 10 targets compared to the other five competitors; it can still beat other fuzzers (i.e., AFL++, FairFuzz, and AFLGo) on five targets when equipped with the static strategy (i.e., the FUNFUZZ-Static). To further understand the unique indirect calls triggered by the 10 studied fuzzers across the 20 repeats, we gather and minimize the resultant sets of indirect calls of each trial and plot the statistics and distributions of indirect calls. Figure 13 visualizes the statistics of covered indirect calls over all 20 trials, where Figure 13(a) reveals the number of the indirect calls triggered in total and Figure 13(b) shows the overlaps of the indirect calls covered by the five studied fuzzers. According to Figure 13(a), FUNFUZZ covers

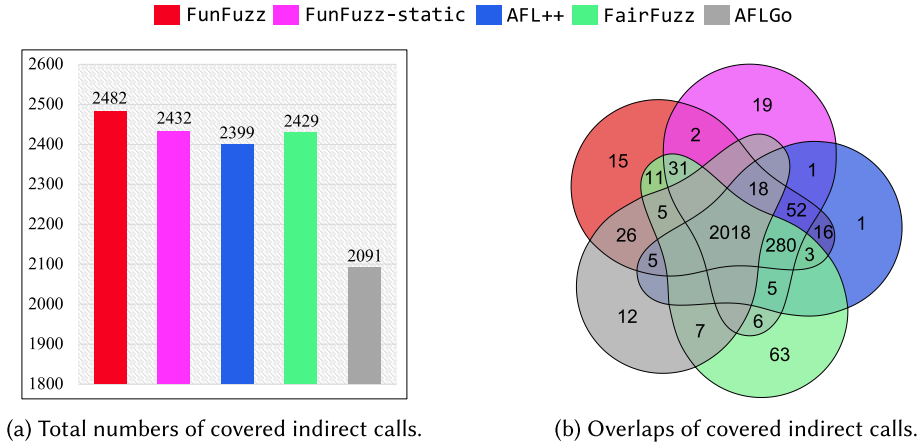


Fig. 13. Total numbers and overlaps of the unique indirect calls covered by FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo on the 10 real-world targets in 20 trials of 24-hour fuzzing.

the most indirect calls and FUNFUZZ-Static, a variant of FUNFUZZ, comes into the second place. Surprisingly, FairFuzz, a fuzzer that built upon the degraded and outdated ancestor of AFL++, beats AFL++ by triggering 30 more indirect calls and appears close behind to FUNFUZZ-Static. Considering that FairFuzz and FUNFUZZ both make use of dynamic execution information, the results may imply that the usage of on-the-fly information benefits the covering of indirect calls, which are also determined at runtime. As illustrated by Figure 13(b), FUNFUZZ, FUNFUZZ-Static, AFL++, FairFuzz, and AFLGo respectively trigger 15, 19, 1, 63, and 12 unique indirect calls that other fuzzers fail to trigger, reflecting a certain degree of complementarity. To sum up, the experimental results demonstrate that our designs will not lower the capability of covering indirect calls concerning the average covered numbers, total covered numbers, and the complementarity of triggered indirect calls.

7.2 Threats to Validity

7.2.1 Threats to Internal Validity. The internal validity correlates with the completeness and fairness of the experimental setups [6, 63]. To handle internal threats, we set the duration of fuzz campaigns to 24 hours to obtain stable coverage results [12, 33], repeat each fuzz campaign for 20 hours to alleviate randomness, and prepare initial seeds following prior works or the mature benchmark [27, 38]. Besides the experiments assessing FUNFUZZ's capabilities of covering code (Section 6.2) and finding bugs (Section 6.3), we conduct experiments exploring the effectiveness of the FS-centered estimation-based power scheduling (Section 6.5) and the impacts of the fun period (Section 6.6), both of which are key components of FUNFUZZ. For all experiments, we carefully document their setups in detail. All these efforts can largely address the threats to internal validity.

7.2.2 Threats to External Validity. The external validity is about the generality of the study results [6]. As for this paper, the threats to external validity comprise two aspects. The first threat lies in the adoption of fuzz targets. Currently, we evaluate FUNFUZZ on 15 fuzz: 10 from seven real-world projects and five from the mature benchmark Magma. The selected fuzz targets are widely adopted in the fuzzing research and include various input formats (i.e., XML, ELF, PDF, PNG, etc.) [39, 41, 63, 83]. As such, the selected targets can provide reliable results for our experiments and analyses. The second threat lies in the selection of comparison objects. We prototype FUNFUZZ on top of AFL++, which is a SOTA fuzzer widely adopted by the fuzzing community, and compare

FUNFUZZ to AFL++ in our experiment. In addition to AFL++, we include SOTA fuzzers like Entropic and K-Scheduler in our evaluation for better comparisons. The results obtained from large-scale experiments and in-depth analyses can greatly demonstrate the competitiveness of FUNFUZZ.

7.2.3 Threats in Design and Implementation. Certain designs and implementations may incur threats to validity [29], and it is scarcely impossible for a technique to be perfect in all aspects. In this paper, we conduct FS computation on top of call graphs constructed with compile-time program information, which may bring threats to the identification of indirect calls. We discuss the threats of indirect calls with experiments in Section 7.1, where the results reveal that, instead of undermining the effectiveness of FUNFUZZ in covering indirect calls, our designs that are built upon compile-time call graphs improve the capability of covering indirect calls regarding the averaged numbers of the covered indirect calls and the total number of the triggered indirect calls. However, the discussed experiment cannot evaluate the volume of code hidden behind the indirect calls, which still remain some threats and are worth further investigation. Since the handling of indirect calls is not the main focus of this paper, we leave it as future work. In addition to discussing indirect calls, we empirically evaluate and analyze the key components of FUNFUZZ, namely the dynamic FS computation (Sections 6.2–6.4), the FS-centred estimation-based power scheduling (Section 6.5), and the lazy FS computation mechanism (Section 6.6). We believe these discussions can confirm the reliability of FUNFUZZ to a large extent.

8 Related Work

8.1 Improving Greybox Fuzzing

Greybox fuzzing is a simple yet effective technique for discovering potential problems in software systems [42, 50, 60, 85]. Although powerful in finding bugs, greybox fuzzing still has great room for improvement. Recently, many works have emerged to facilitate greybox fuzzing from various perspectives, such as initial seed selection [28, 65], seed scheduling [72, 77], input generation mechanisms [47, 79], execution guidance [31, 35, 36, 39], and power scheduling [9, 14, 71, 80]. For example, to better choose initial seeds for greybox fuzzing, Herrera et al. [28] first perform a large-scale study to investigate how initial seeds affect the performance of greybox fuzzing, and then propose a constraint-based technique to help users minimize seed corpus with an EvalMaxSAT solver. To better improve the efficiency of exposing vulnerabilities, Wang et al. [77] evaluate coverage by security impacts and prioritize seeds that are most security-relevant. You et al. [79] propose a technique named SLF to facilitate fuzzing under the scenario where valid seeds are missing. Kukucka et al. [35] make use of concolic guidance to identify and reuse interesting bytes to improve the effectiveness of greybox fuzzing.

In this paper, we propose an FS-centered estimation-based power scheduling that assigns energy to seeds according to significance scores. Among all the topics on facilitating greybox fuzzing, the refining of power scheduling is the most related to our work. Works like FairFuzz [38] and AFLFast [11] belong to this taxonomy. Specifically, FairFuzz is equipped with a two-pronged mechanism to identify rare branches and schedule the seeds to drive executions toward hard-to-reach code regions. AFLFast improves the efficiency of greybox fuzzing by profiling the frequencies of seen paths and gravitating executions towards the low-frequency ones. We have included FairFuzz and AFLFast (which is the default power scheduling built in AFL++ version 4.04c) in our evaluation as they both guide fuzzing with dynamic information. Besides FairFuzz and AFLFast, there are many other works on power scheduling. For example, Böhme et al. [9] facilitate greybox fuzzing by steering executions toward revealing more information. Yue et al. [80] incorporate greybox fuzzing with reinforcement learning and leverage a variant of the adversarial multi-armed bandit algorithm

to assign more energy to seeds that execute low-frequency paths. In contrast to FUNFUZZ, these works ignore the significant degrees of functions and treat the executed functions equally.

8.2 Software Testing with Graph Analysis

Graphs are an important representation in software testing [5], and graph analysis is widely adopted in many software testing research such as mutation testing [25], test selection [37], test prioritization [78], and oracle construction [15]. There is also fuzzing research that exploits graph analysis. For example, AFLGo [10] relies on the call graph and control flow graphs extracted from the PUTs to calculate distances to the target locations. WindRanger [19] identifies the deviation basic blocks within the control flow graphs and leverages them to get to the target sites faster. Influence analysis is a common topic of graph analysis and is widely applied to various fields such as finance [67], social network analysis [62], citation analysis [61]. With the aim of exploring code more efficiently, She et al. [72] propose a seed scheduling technique named K-Scheduler to apply influence analysis to edge horizon graphs constructed with a batch of seeds and control flow graphs to prioritize seeds that are most likely to reach fresh basic blocks. Although adopting the same core algorithm, K-Scheduler and FUNFUZZ are orthogonal techniques that hold separate focuses on horizon basic blocks and significant functions.

9 Conclusion

In this paper, we present FUNFUZZ, an FS-aware greybox fuzzing technique that optimizes promising seeds with an estimation-based power scheduling. Our key insight is that the significant degrees of functions, or FS, are unequal in practice, and seeds that executing more significant functions should be focused and optimized. We first present how the significant degrees of functions affect the performance of seeds through an empirical study with 10 real-world programs. Based on the observed empirical results, we then establish FS to quantify the significant degrees of the executed functions. To evaluate FUNFUZZ, we implement FUNFUZZ on top of AFL++ and conduct an extensive study with 15 targets and seven competitor fuzzers. Our extensive study not only demonstrates FUNFUZZ's superiority in covering code and finding bugs but also showcases the effectiveness of FUNFUZZ's individual key components.

Acknowledgment

We would like to thank the anonymous reviewers for their insightful comments.

References

- [1] AFL++-team. 2023. Test Cases Provided by AFL & AFL++. Retrieved March 2023 from <https://github.com/AFLplusplus/AFLplusplus/tree/stable/testcases>
- [2] AFL++-Team. 2024. AFL++ Power Schedules. Retrieved May 20, 2024 from https://aflplusplus.com/docs/power_schedules/
- [3] Charu C. Aggarwal. 2011. *An Introduction to Social Network Data Analytics*. Springer.
- [4] Haldun Akoglu. 2018. User's guide to correlation coefficients. *Turkish Journal of Emergency Medicine* 18, 3 (2018), 91–93.
- [5] Paul Ammann and Jeff Offutt. 2016. *Introduction to Software Testing*. Cambridge University Press.
- [6] Apostolos Ampatzoglou, Stamatia Bibi, Paris Avgeriou, Marijn Verbeek, and Alexander Chatzigeorgiou. 2019. Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. *Information and Software Technology* 106 (2019), 201–230.
- [7] Jinsheng Ba, Marcel Böhme, Zahra Mirzamomen, and Abhik Roychoudhury. 2022. Stateful greybox fuzzing. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*, 3255–3272.
- [8] Nicholas H. Bingham and John M. Fry. 2010. *Regression: Linear Models in Statistics*. Springer Science & Business Media.
- [9] Marcel Böhme, Valentin J. M. Manès, and Sang Kil Cha. 2020. Boosting fuzzer efficiency: An information theoretic perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE '20)*, 678–689.

- [10] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, 2329–2344.
- [11] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2017. Coverage-based greybox fuzzing as markov chain. *IEEE Transactions on Software Engineering* 45, 5 (2017), 489–506.
- [12] Marcel Böhme, László Szekeres, and Jonathan Metzman. 2022. On the reliability of coverage-based fuzzer benchmarking. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*, 1621–1633.
- [13] Brad Calder and Dirk Grunwald. 1994. Reducing indirect function call overhead in C++ programs. In *Proceedings of the 21st ACM SIGPLAN-SIGACT symposium on Principles of Programming Languages*, 397–408.
- [14] Hongxu Chen, Shengjian Guo, Yinxing Xue, Yulei Sui, Cen Zhang, Yuekang Li, Haijun Wang, and Yang Liu. 2020. MUZZ: Thread-aware grey-box fuzzing for effective bug hunting in multithreaded programs. In *Proceedings of the 29th USENIX Security Symposium (Security '20)*, 2325–2342.
- [15] Junjie Chen, Yanwei Bai, Dan Hao, Lingming Zhang, Lu Zhang, Bing Xie, and Hong Mei. 2016. Supporting oracle construction via static analysis. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE '16)*, 178–189.
- [16] Clang-team. 2023. Clang AddressSanitizer. Retrieved March 2023 from <https://clang.llvm.org/docs/AddressSanitizer.html>
- [17] Clang-team. 2024. LLVM Source Code Coverage. Retrieved May 2024 from <https://clang.llvm.org/docs/SourceBasedCodeCoverage.html>
- [18] Israel Cohen, Yiteng Huang, Jingdong Chen, Jacob Benesty, Jacob Benesty, Jingdong Chen, Yiteng Huang, and Israel Cohen. 2009. Pearson correlation coefficient. In *Noise Reduction in Speech Processing*. Springer, 1–4.
- [19] Zhengjie Du, Yuekang Li, Yang Liu, and Bing Mao. 2022. Windranger: A directed greybox fuzzer driven by deviation basic blocks. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*, 2440–2451.
- [20] Dynamorio-team. 2024. Dynamorio: Dynamic Instrumentation Tool Platform. Retrieved August 2024 from <https://dynamorio.org/>
- [21] Chunrong Fang, Zixi Liu, Yangyang Shi, Jeff Huang, and Qingkai Shi. 2020. Functional code clone detection with syntax and semantics fusion learning. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '20)*, 516–527.
- [22] Julian J. Faraway. 2014. *Linear Models with R*. CRC Press.
- [23] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: Combining incremental steps of fuzzing research. In *Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT '20)*.
- [24] FunFuzz-team. 2024. Artifacts of FuzzFuzz. Retrieved August 2024 from <https://github.com/funfuzz-tosem2023/funfuzz-artifacts>
- [25] Aayush Garg, Milos Ojdanic, Renzo Degiovanni, Thierry Titchou Chekam, Mike Papadakis, and Yves Le Traon. 2022. Cerebro: Static subsuming mutant selection. *IEEE Transactions on Software Engineering* 49, 1 (2022), 24–43.
- [26] Emre Güler, Sergej Schumilo, Moritz Schloegel, Nils Bars, Philipp Görz, Xinyi Xu, Cemal Kaygusuz, and Thorsten Holz. 2024. Atropos: Effective fuzzing of web applications for server-side vulnerabilities. In *Proceedings of the International Conference on USENIX Security Symposium*.
- [27] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A ground-truth fuzzing benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3 (2020), 1–29.
- [28] Adrian Herrera, Hendra Gunadi, Shane Magrath, Michael Norrish, Mathias Payer, and Antony L. Hosking. 2021. Seed selection for successful fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21)*, 230–243.
- [29] Adrian Herrera, Mathias Payer, and Antony L. Hosking. 2023. DataFlow: Toward a data-flow-guided fuzzer. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 1–31.
- [30] HexHive. 2024. Magma Bug List. Retrieved January 2024 from <https://hexhive.epfl.ch/magma/docs/bugs.html>
- [31] Heqing Huang, Yiyuan Guo, Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. 2022. BEACON: Directed grey-box fuzzing with provable path pruning. In *Proceedings-2022 IEEE Symposium on Security and Privacy (SP '22)*.
- [32] Leo Katz. 1953. A new status index derived from sociometric analysis. *Psychometrika* 18, 1 (1953), 39–43.
- [33] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating fuzz testing. In *Proceedings of the ACM SIGSAC Conference On Computer and Communications Security*, 2123–2138.
- [34] Vijay Kotu and Bala Deshpande. 2018. *Data Science: Concepts and Practice*. Morgan Kaufmann.
- [35] James Kukucka, Luis Pina, Paul Ammann, and Jonathan Bell. 2022. CONFETTI: Amplifying concolic guidance for fuzzers. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*, 438–450.
- [36] Gwangmu Lee, Woochul Shim, and Byoungyoung Lee. 2021. Constraint-guided Directed Greybox Fuzzing. In *Proceedings of the 30th USENIX Security Symposium (Security '21)*, 3559–3576.
- [37] Owolabi Legunsen, August Shi, and Darko Marinov. 2017. STARTS: STAtic regression test selection. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE '17)*, IEEE, 949–954.

- [38] Caroline Lemieux and Koushik Sen. 2018. Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18)*, 475–485.
- [39] Shaohua Li and Zhendong Su. 2023. Accelerating fuzzing through prefix-guided execution. *Proceedings of the ACM on Programming Languages (OOPSLA)* OOPSLA, 1–27.
- [40] Yuchen Li, Ju Fan, Yanhao Wang, and Kian-Lee Tan. 2018. Influence maximization on social graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering* 30, 10 (2018), 1852–1872.
- [41] Hongliang Liang, Xinglin Yu, Xianglin Cheng, Jie Liu, and Jin Li. 2023. Multiple targets directed greybox fuzzing. *IEEE Transactions on Dependable and Secure Computing* 21, 1 (2023), 3251–15.
- [42] Jie Liang, Yu Jiang, Mingzhe Wang, Xun Jiao, Yuanliang Chen, Houbing Song, and Kim-Kwang Raymond Choo. 2021. DeepFuzzer: Accelerated deep greybox fuzzing. *IEEE Transactions on Dependable and Secure Computing* 18, 6 (2021), 2675–2688.
- [43] LibFuzzer. 2023. Libfuzzer: Coverage Guided Fuzz Testing. Retrieved June 6, 2022 from <https://llvm.org/docs/LibFuzzer.html>
- [44] Xiang Lin, Tao Shang, and Jianwei Liu. 2014. An estimation method for relationship strength in weighted social network graphs. *Journal of Computer and Communications* 2, 4 (2014), 82.
- [45] LLVM-team. 2023. Introduction LLVM pass. Retrieved August 12, 2022 from <https://llvm.org/docs/WritingAnLLVMPass.html>
- [46] Kangjie Lu and Hong Hu. 2019. Where does it go? Refining indirect-call targets with multi-layer type analysis. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 1867–1881.
- [47] Chenyang Lyu, Shouling Ji, Chao Zhang, Yuwei Li, Wei-Han Lee, Yu Song, and Raheem Beyah. 2019. Mopt: Optimized mutation scheduling for fuzzers. In *Proceedings of the 28th USENIX Security Symposium (Security '19)*, 1949–1966.
- [48] Hexhive Magma. 2023. List of the Fuzz Targets from Magma. Retrieved December 21, 2023 from <https://hexhive.epfl.ch/magma/>
- [49] Magma-Community. 2024. Usage of Magma llvm_cov. Retrieved April 2024 from <https://github.com/HexHive/magma/issues/144>
- [50] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2019. The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering* 47, 11 (2019), 2312–2331.
- [51] Dastan Maulud and Adnan M. Abdulazeez. 2020. A review on linear regression comprehensive in machine learning. *Journal of Applied Science and Technology Trends* 1, 4 (2020), 140–147.
- [52] Patrick E. McKnight and Julius Najab. 2010. Mann-Whitney U test. *The Corsini Encyclopedia of Psychology*. John Wiley & Sons, Ltd, New York, 1–1.
- [53] Jonathan Metzmann, László Szekeres, Laurent Simon, Read Sprabery, and Abhishek Arya. 2021. Fuzzbench: An open fuzzer benchmarking platform and service. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE '21)*, 1393–1403.
- [54] Barton P. Miller, Louis Fredriksen, and Bryan So. 1990. An empirical study of the reliability of unix utilities. *Communications of the ACM* 33, 12 (1990), 32–44.
- [55] MJS-team. 2022. Embedded Javascript Engine for C/c++. Retrieved from <https://github.com/cesanta/mjs>
- [56] Eisha Nathan and David A. Bader. 2017. A dynamic algorithm for updating katz centrality in graphs. In *Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*, 149–154.
- [57] Networkit. 2023. Python Library for Large-Scale Network Analysis. Retrieved from <https://networkit.github.io/>
- [58] M. E. J. Newman. 2016. Mathematics of Networks.
- [59] Hoang Lam Nguyen and Lars Grunske. 2022. BeDivFuzz: Integrating behavioral diversity into generator-based fuzzing. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*, 249–261.
- [60] Rohan Padhye, Caroline Lemieux, Koushik Sen, Laurent Simon, and Hayawardh Vijayakumar. 2019. Fuzzfactory: Domain-specific fuzzing with waypoints. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–29.
- [61] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The Pagerank Citation Ranking: Bringing Order to the Web*. Technical Report. Stanford InfoLab.
- [62] Sancheng Peng, Yongmei Zhou, Lihong Cao, Shui Yu, Jianwei Niu, and Weijia Jia. 2018. Influence analysis in social networks: A survey. *Journal of Network and Computer Applications* 106 (2018), 17–32.
- [63] Ruixiang Qian, Qianjun Zhang, Chunrong Fang, Ding Yang, Shun Li, Binyu Li, and Zhenyu Chen. 2024. DiPri: Distance-based seed prioritization for greybox fuzzing. *ACM Transactions on Software Engineering and Methodology* (Mar. 2024). DOI: <https://doi.org/10.1145/3654440>
- [64] Timothy R. C. Read and Noel A. C. Cressie. 2012. *Goodness-of-Fit Statistics for Discrete Multivariate Data*. Springer Science & Business Media.

- [65] Alexandre Rebert, Sang Kil Cha, Thanassis Avgerinos, Jonathan Foote, David Warren, Gustavo Grieco, and David Brumley. 2014. Optimizing seed selection for fuzzing. In *Proceedings of the 23rd USENIX Security Symposium (Security '14)*, 861–875.
- [66] Alvin C. Rencher and G. Bruce Schaalje. 2008. *Linear Models in Statistics*. John Wiley & Sons.
- [67] Eduardo J. Ruiz, Vagelis Hristidis, Carlos Castillo, Aristides Gionis, and Alejandro Jaimes. 2012. Correlating financial time series with micro-blogging activity. In *Proceedings of the 5th ACM International Conference on Web Search and Data Mining*, 513–522.
- [68] Sergej Schumilo, Cornelius Aschermann, Andrea Jemmett, Ali Abbasi, and Thorsten Holz. 2022. Nyx-Net: Network fuzzing with incremental snapshots. In *Proceedings of the 17th European Conference on Computer Systems (EuroSys '22)*. DOI: <https://doi.org/10.1145/3492321.3519591>
- [69] Kostya Serebryany. 2017. Oss-fuzz: Google's Continuous Fuzzing Service for Open Source Software.
- [70] Dongdong She, Rahul Krishna, Lu Yan, Suman Jana, and Baishakhi Ray. 2020. MTFuzz: Fuzzing with a multi-task neural network. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE '20)*, 737–749.
- [71] Dongdong She, Kexin Pei, Dave Epstein, Junfeng Yang, Baishakhi Ray, and Suman Jana. 2019. Neuzz: Efficient fuzzing with neural program smoothing. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'19)*, 803–817.
- [72] Dongdong She, Abhishek Shah, and Suman Jana. 2022. Effective seed scheduling for fuzzing with graph centrality analysis. In *Proceedings of IEEE Symposium on Security and Privacy (SP '22)*.
- [73] Walid Taha. 2004. A gentle introduction to multi-stage programming. In *Proceedings of the International Conference on Domain-Specific Program Generation: International Seminar*. Revised Papers. Springer, 30–50.
- [74] Ari Takanen, Jared D. Demott, Charles Miller, and Atte Kettunen. 2018. *Fuzzing for Software Security Testing and Quality Assurance*. Artech House.
- [75] Tian Tan, Yue Li, Xiaoxing Ma, Chang Xu, and Yannis Smaragdakis. 2021. Making pointer analysis more precise by unleashing the power of selective context sensitivity. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–27.
- [76] Marno Verbeek. 2008. *A guide to modern econometrics*. John Wiley & Sons.
- [77] Yanhao Wang, Xiangkun Jia, Yuwei Liu, Kyle Zeng, Tiffany Bao, Dinghao Wu, and Purui Su. 2020. Not all coverage measurements are equal: Fuzzing by coverage accounting for input prioritization. In *Proceedings of the International Conference on the Network and Distributed System Security Symposium (NDSS '20)*, 1–17.
- [78] Zan Wang, Hanmo You, Junjie Chen, Yingyi Zhang, Xuyuan Dong, and Wenbin Zhang. 2021. Prioritizing test inputs for deep neural networks via mutation analysis. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*. IEEE, 397–409.
- [79] Wei You, Xuwei Liu, Shiqing Ma, David Perry, Xiangyu Zhang, and Bin Liang. 2019. Slf: Fuzzing without valid seed inputs. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE '19)*, 712–723.
- [80] Tai Yue, Pengfei Wang, Yong Tang, Enze Wang, Bo Yu, Kai Lu, and Xu Zhou. 2020. EcoFuzz: Adaptive energy-saving greybox fuzzing as a variant of the adversarial multi-armed bandit. In *Proceedings of the 29th USENIX Security Symposium (Security '20)*, 2307–2324.
- [81] Michal Zalewski. 2023. AFL Vulnerability Trophy Case. Retrieved March 2023 from <https://lcamtuf.coredump.cx/afl/#bugs>
- [82] Michal Zalewski. 2023. American Fuzzy Lop GitHub Repository. Retrieved March 2023 from <https://github.com/google/AFL>
- [83] Kunpeng Zhang, Xi Xiao, Xiaogang Zhu, Ruoxi Sun, Minhui Xue, and Sheng Wen. 2022. Path transitions tell more: Optimizing fuzzing schedules via runtime program states. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*, 1658–1668.
- [84] Han Zheng, Jiayuan Zhang, Yuhang Huang, Zezhong Ren, He Wang, Chunjie Cao, Yuqing Zhang, Flavio Toffalini, and Mathias Payer. 2023. FISHFUZZ: Catch deeper bugs by throwing larger nets. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 1343–1360. Retrieved from <https://www.usenix.org/conference/usenixsecurity23/presentation/zheng>
- [85] Xiaogang Zhu, Sheng Wen, Seyit Camtepe, and Yang Xiang. 2022. Fuzzing: A survey for roadmap. *ACM Computing Surveys* 54, 11s (2022), 1–36.

Received 12 September 2023; revised 10 August 2024; accepted 28 September 2024